Freque Plugin Development Guide v2.5

Complete guide for creating audio-reactive visualizations with advanced Three.js and WebGL techniques

What's New in v2.5

- **Window Resize Handling** - Complete guide to responsive canvas sizing and game state preservation ⭐ NEW
- **Audio Integration Clarification** - Clear documentation on how to properly access audio data ⭐ NEW
- **Dropdown Format Requirements** - Complete guide with correct format examples ⭐ NEW
- **Control Variable Initialization** - Critical pattern for preventing undefined variables ⭐ NEW
- **Making Controls Work** - Ensuring controls actually affect visualization ⭐ NEW
- **Audio Reactivity Best Practices** - Base value + multiplier pattern for responsive controls ⭐ NEW
- **Common Mistakes Section** - Side-by-side wrong vs. correct examples ⭐ NEW

What’s New in v2.4

- **Ring Spawning System** - Efficient pattern for continuous tunnel effects ⭐ NEW
- **Post-Processing Effects** - Pixelation and color banding for retro aesthetics ⭐ NEW
- **Advanced Audio Reactivity** - Granular controls with sensitivity, smoothing, and per-feature intensity ⭐ NEW
- **Frequency-Specific Reactivity** - Map bass/mid/treble to specific visual effects ⭐ NEW
- **Custom Dial Fill Colors** - Per-plugin dial color customization using CSS variables ⭐ NEW

What’s New in v2.3

- **Enhanced Retro Effects** - Improved VHS glitch, RGB shift with vertical component ⭐ NEW
- **Film Grain Static** - Clustered noise with color artifacts for authenticity ⭐ NEW
- **Extended Color Palettes** - 10 retro color schemes (C64, Game Boy, CRT, etc.) ⭐ NEW

What’s New in v2.2

- **WebGL/WebGL2 Plugin Architecture** - Raw WebGL and WebGL2 shader plugins now supported ✅
- **Context Type Flexibility** - Plugins can choose 2D, WebGL, or WebGL2 contexts
- **Lazy Context Loading** - Base class no longer forces 2D context creation
- **Backwards Compatible** - Existing 2D and Three.js plugins work unchanged

What's in v2.1

- **Float Speed Non-Linear Scaling** - Two-part curve for better UX
- **Video Intensity Auto-Adjustment** - Automatic brightness compensation
- **Control Update Behavior** - Documentation of immediate vs deferred updates

What's in v2.0

- **Cube Camera Video Reflection System** - Complete implementation guide
- **Layer Management** - Selective visibility for complex scenes
- **Color Space Management** - sRGB encoding best practices
- **Dynamic Environment Mapping** - Position-based reflections
- **Performance Optimization** - Strategies for complex 3D scenes

---

Table of Contents

1. Quick Start
2. Canvas Context Types ⭐ NEW
3. Window Resize Handling ⭐ NEW
4. 2D Canvas Plugins
5. WebGL/WebGL2 Plugins ⭐ NEW
6. Three.js Integration
7. Advanced Three.js Techniques
8. Audio Integration
9. Advanced Audio Reactivity ⭐ NEW
10. Ring Spawning System ⭐ NEW
11. Post-Processing Effects ⭐ NEW
12. UI Controls

13. Performance Optimization

14. Best Practices

Quick Start

Basic Plugin Template

```
class MyPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myplugin', visualizer, {
            version: '1.0.0',
            author: 'Your Name',
            description: 'My Awesome Plugin',
            targetFPS: 60,
            dialFillColor: '--accent-color' // Optional: Custom dial fill color (CSS vari
        });

        this.setupControls();
        this.setupPresets();
    }

    setupControls() {
        this.addControl('size', {
            type: 'dial',
            label: 'Size',
            min: 1,
            max: 100,
            step: 1,
            value: 50,
            onChange: (value) => {
                this.size = value;
            }
        });
    }

    setupPresets() {
        this.addPreset('default', {
            name: 'Default',
            values: { size: 50 }
        });
    }
```

```
    onInitialize() {
        // Setup your visualization
    }

    onUpdate(deltaTime, timestamp, sharedAudioData) {
        // Update logic
        const energy = this.getAudioEnergy();
    }

    onRender(deltaTime, timestamp, sharedAudioData) {
        // Draw to canvas
        this.ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);
    }
}

// Auto-register
setTimeout(() => {
    if (window.visualizer && window.FrequePluginBase) {
        new MyPlugin(window.visualizer);
    }
}, 500);
```

Canvas Context Types

Understanding Canvas Context Architecture

CRITICAL CONCEPT: A canvas element can only have ONE context type at a time. Once you call `canvas.getContext('2d')`, you cannot get a WebGL or WebGL2 context from that same canvas. This is a fundamental browser limitation.

Previous Architecture (Before v2.2)

```
// In FrequePluginBase resize/setCanvasDimensions:
this.canvas.width = width;
this.canvas.height = height;
this.ctx = this.canvas.getContext('2d'); // âŒ FORCED 2D context creation
```

Problem: This forced EVERY plugin to use a 2D context, making WebGL/WebGL2 plugins impossible.

New Architecture (v2.2+)

```
// 1. In constructor - Lazy-loaded 2D context
this._ctx = null; // Private storage

Object.defineProperty(this, 'ctx', {
    get: function() {
        if (!this._ctx && this.canvas) {
            this._ctx = this.canvas.getContext('2d'); // Only created when accessed
        }
        return this._ctx;
    }
});

// 2. In setCanvasDimensions - NO context creation
this.canvas.width = width;
this.canvas.height = height;
// Context is NOT created - plugin decides what it needs

// 3. In render() - Smart clearing
if (this.shouldClearCanvas() && this._ctx) { // Only clears if 2D context exists
    this.ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);
}
```

Solution: The base class no longer creates a context automatically. Plugins explicitly create the context type they need.

Plugin Type Decision Guide

Use 2D Canvas When:

- Drawing shapes, text, images, gradients
- Using canvas drawing API (fillRect, drawImage, etc.)
- Simple 2D visualizations
- Particle systems in 2D space

Example Plugins: Waveform displays, 2D particle effects, spectrum analyzers

---

Use WebGL When:

- Need GPU-accelerated 2D effects
- Custom shaders for post-processing
- Large numbers of simple objects (thousands+)
- Compatibility with older browsers/devices

Example Plugins: GPU particle systems, shader-based effects

Use WebGL2 When:

- Advanced shader features (texture arrays, transform feedback)
- Modern GPU capabilities required
- Complex fragment shader calculations
- Better performance on modern hardware

Example Plugins: Advanced liquid simulations, complex procedural effects, PSYCH visualizer

---

Use Three.js When:

- 3D scenes with meshes, lights, cameras
- Video reflections (cube cameras)
- Complex 3D geometry
- Environment mapping

Example Plugins: Chrome Spheres, 3D objects, particle clouds in 3D space

---

Context Type Compatibility Matrix

Feature	2D Canvas	WebGL	WebGL2	Three.js
Browser Support	âœ… Universal	âœ… ~97%	âœ… ~95%	âœ… ~97%
GPU Acceleration	âŒ CPU Only	âœ… Yes	âœ… Yes	âœ… Yes
Custom Shaders	âŒ No	âœ… GLSL 1.0	âœ… GLSL 3.0	âœ… GLSL 1.0/3.0
3D Rendering	âŒ No	âš ï¸ Manual	âš ï¸ Manual	âœ… Built-in
Learning Curve	âœ… Easy	âš ï¸ Moderate	âš ï¸ Hard	âœ… Moderate
Performance	âš ï¸ Limited	âœ… Good	âœ… Excellent	âœ… Good
File Size	âœ… Minimal	âœ… Small	âœ… Small	âš ï¸ ~700KB

Window Resize Handling

Overview: The Resize Architecture

Freque implements a three-tier resize system to ensure all plugins resize smoothly without flickering:

1. **Global Resize Listener** (`main.js`) - Detects window size changes, throttles events
2. **Base Class Handler** (`FrequePluginBase`) - Updates canvas dimensions, calls plugin hooks
3. **Plugin-Specific Handler** (`onResize()`) - Recreates/repositions plugin-specific elements

---

The Complete Resize Flow

1. Global Resize Listener (main.js)

When the browser window is resized, a throttled event handler manages the update:

```
// In main.js
let pluginResizeTimeout;
window.addEventListener('resize', () => {
    // Native visualizations resize immediately (smooth handling)
    if (window.visualizer && window.visualizer.infiniteZoom) {
        window.visualizer.infiniteZoom.resize();
    }
    // ... other native visualizations

    // Throttle plugin resizes to prevent flickering during drag-resize
    clearTimeout(pluginResizeTimeout);
    pluginResizeTimeout = setTimeout(() => {
        if (window.pluginManager && window.pluginManager.plugins) {
            window.pluginManager.plugins.forEach(plugin => {
                if (plugin && plugin.resize) {
                    plugin.resize();  // Calls FrequePluginBase.resize()
                }
            });
        }
    }, 150); // Wait 150ms after resize stops
});
```

Why Throttling? - **Without throttle**: Plugins resize on every pixel change → **visible flickering** during window drag - **With throttle**: Plugins resize once after drag stops → **smooth experience** - **150ms delay**: Optimal balance between responsiveness and flicker prevention

---

2. Base Class Handler (FrequePluginBase)

The base class provides automatic canvas dimension updates:

```
// In plugin-base.js
resize() {
    if (!this.canvas) return;

    // Get container dimensions (NOT canvas.clientWidth)
    const container = document.getElementById('visualizationContainer');
    if (!container) return;

    const rect = container.getBoundingClientRect();

    // Handle edge case: container not visible yet
    if (rect.width === 0 || rect.height === 0) {
        const fallbackWidth = Math.max(window.innerWidth, 800);
        const fallbackHeight = Math.max(window.innerHeight, 600);
        this.setCanvasDimensions(fallbackWidth, fallbackHeight);
        return;
    }

    this.setCanvasDimensions(rect.width, rect.height);
}

setCanvasDimensions(width, height) {
    if (!this.canvas) return;

    // Update canvas buffer size
    this.canvas.width = width;
    this.canvas.height = height;

    // Call plugin-specific resize handler
    if (this.onResize) {
        this.onResize(width, height);
    }
}
```

Critical: Canvas CSS Styling

The base class sets canvas CSS in `createCanvas()`:

```
createCanvas() {
    this.canvas = document.createElement('canvas');
    // ... canvas setup

    // CRITICAL: Percentage sizing ensures responsive scaling
    this.canvas.style.width = '100%';
    this.canvas.style.height = '100%';

    return this.canvas;
}
```

Why Percentage CSS? - ✅ Canvas display size scales with container automatically - ✅ Works correctly with window resize without manual CSS updates - ✅ Maintains aspect ratio properly - ❌ Fixed pixel CSS (`width: 800px`) doesn't scale with window

---

3. Plugin-Specific Handler (onResize)

Each plugin implements `onResize()` to handle its specific resize needs:

```
class MyPlugin extends FrequePluginBase {
    onResize(width, height) {
        // STEP 1: Update positioning variables
        this.centerX = width / 2;
        this.centerY = height / 2;

        // STEP 2: Recreate size-dependent game objects
        this.recreateParticles();
        this.recreateGrid();

        // STEP 3: Reposition existing objects
        this.player.x = width / 2;
        this.player.y = height / 2;

        // STEP 4: Preserve game state (score, lives, etc.)
        // These are NOT reset during resize
    }
}
```

Real-World Example: Alienz Plugin Resize

The Alienz plugin demonstrates proper resize handling for a game with multiple elements:

```
onResize(width, height) {
    // Update ship position for new canvas size
    this.shipX = width / 2;
    this.shipTargetX = width / 2;

    // Recreate barriers for new size
    this.createBarriers();

    // Recreate aliens for new size (preserves score/wave)
    this.recreateAliens();

    // Recreate stars for new canvas dimensions
    this.createStars();
}

// Helper method that preserves game state
recreateAliens() {
    // Preserve score and wave when recreating aliens
    const currentScore = this.score;
    const currentWave = this.wave;
    this.createAliens();
    this.score = currentScore;  // Restore after recreation
    this.wave = currentWave;
}
```

What This Achieves: 1. **Ship** → Repositioned to center of new canvas 2. **Barriers** → Recreated at correct positions for new width 3. **Aliens** → Recreated in proper grid for new canvas size (game state preserved) 4. **Stars** → Regenerated to fill new canvas dimensions 5. **Score/Wave/Lives** → Preserved throughout resize

---

Critical Patterns for Different Plugin Types

For 2D Canvas Game Plugins

```
onResize(width, height) {
    // 1. Update center point
    this.centerX = width / 2;
    this.centerY = height / 2;

    // 2. Reposition player/ship
    this.player.x = width / 2;
    this.player.y = height / 2;

    // 3. Recreate grid-based elements (aliens, barriers, etc.)
    this.recreateEnemies();
    this.recreateObstacles();

    // 4. Recreate background elements (stars, particles)
    this.recreateBackground();

    // 5. Adjust boundaries for collision detection
    this.leftBoundary = 50;
    this.rightBoundary = width - 50;
    this.topBoundary = 50;
    this.bottomBoundary = height - 50;
}
```

For Three.js Plugins

```
onResize(width, height) {
    // 1. Update camera aspect ratio
    if (this.camera) {
        this.camera.aspect = width / height;
        this.camera.updateProjectionMatrix();
    }

    // 2. Update renderer size
    if (this.renderer) {
        this.renderer.setSize(width, height);
    }

    // 3. Reposition 3D objects if needed
    this.updateObjectPositions();
}
```

For WebGL/WebGL2 Plugins

```
onResize(width, height) {
    if (!this.gl) return;

    // 1. Update canvas buffer size with DPI
    const dpr = window.devicePixelRatio || 1;
    this.canvas.width = width * dpr;
    this.canvas.height = height * dpr;

    // 2. Update WebGL viewport
    this.gl.viewport(0, 0, this.canvas.width, this.canvas.height);

    // 3. Update resolution uniform for shaders
    if (this.program && this.uniforms.resolution) {
        this.gl.uniform2f(this.uniforms.resolution, this.canvas.width, this.canvas.height
    }
}
```

For Particle System Plugins

```
onResize(width, height) {
    // 1. Update emission point
    this.emitterX = width / 2;
    this.emitterY = height / 2;

    // 2. Update particle boundaries
    this.maxX = width;
    this.maxY = height;

    // 3. Remove off-screen particles
    this.particles = this.particles.filter(p => {
        return p.x >= 0 && p.x <= width && p.y >= 0 && p.y <= height;
    });

    // 4. Keep existing particles (don't recreate entire system)
}
```

Common Resize Issues and Solutions

Issue 1: Elements Disappear After Resize

Problem:

```
// ❌ BAD: Only updating canvas size, not recreating elements
onResize(width, height) {
    // Canvas is resized by base class, but aliens are still at old positions!
}
```

Solution:

```
// ✅ GOOD: Recreate or reposition all elements
onResize(width, height) {
    this.centerX = width / 2;
    this.centerY = height / 2;
    this.recreateAliens();  // Recreate at new positions
    this.recreateBarriers();
    this.player.x = width / 2;  // Reposition player
}
```

Issue 2: Game State Lost on Resize

Problem:

```
// ❌ BAD: Calling createAliens() resets score/wave
onResize(width, height) {
    this.createAliens();  // This resets game state!
}
```

Solution:

```
// ✅ GOOD: Use helper method that preserves state
recreateAliens() {
    const currentScore = this.score;
    const currentWave = this.wave;
    const currentLives = this.lives;

    this.createAliens();  // Recreate aliens

    // Restore game state
    this.score = currentScore;
    this.wave = currentWave;
    this.lives = currentLives;
}

onResize(width, height) {
    this.recreateAliens();  // Use state-preserving method
}
```

Issue 3: Flickering During Window Drag

Problem: Plugin resizes on every single pixel change during window drag.

Solution: This is handled automatically by the throttled resize in `main.js`. Plugins don't need to do anything special - the 150ms delay ensures smooth resizing.

Issue 4: Canvas Stretched/Distorted

Problem:

```
// ❌ BAD: Using fixed pixel CSS
this.canvas.style.width = '800px';
this.canvas.style.height = '600px';
```

Solution:

```
// ✅ GOOD: Using percentage CSS (already set by base class)
this.canvas.style.width = '100%';
this.canvas.style.height = '100%';
```

Note: The base class already sets percentage CSS in `createCanvas()`, so you don't need to do this manually.

Issue 5: WebGL Canvas Appears Blurry After Resize

Problem: Not accounting for device pixel ratio after resize.

Solution:

```
onResize(width, height) {
    if (!this.gl) return;

    // Account for high-DPI displays
    const dpr = window.devicePixelRatio || 1;
    this.canvas.width = width * dpr;
    this.canvas.height = height * dpr;

    // Update viewport to match
    this.gl.viewport(0, 0, this.canvas.width, this.canvas.height);
}
```

Checklist for Resize Implementation

When implementing `onResize()` in your plugin, ensure:

Required Steps

- ☐ Update center point variables (`centerX` , `centerY`)
- ☐

Reposition player/ship/main character

- ☐ Recreate grid-based or size-dependent elements
- ☐ Recreate background elements (stars, particles, etc.)
- ☐ Update boundaries for collision detection

Game State Preservation

- ☐ Score is preserved across resize
- ☐ Lives/health is preserved
- ☐ Wave/level is preserved
- ☐ Power-ups/inventory is preserved
- ☐ Use helper methods that explicitly save/restore state

Canvas & Context

- ☐ Don’t manually set canvas width/height (base class handles it)
- ☐ Don’t manually set CSS styling (base class handles it)
- ☐ For WebGL: Update viewport with `gl.viewport()`
- ☐ For Three.js: Update camera aspect ratio

Testing

- ☐ Test resizing to wider window
- ☐ Test resizing to taller window
- ☐ Test resizing to very small window
- ☐ Test during active gameplay (elements moving, score changing)
- ☐

Test that game state persists through resize

Best Practices

✅ DO:

1. **Recreate size-dependent elements**

```
this.recreateAliens();
this.recreateBarriers();
```

2. **Preserve game state explicitly**

```
const state = this.getGameState();
this.recreate();
this.setGameState(state);
```

3. **Update positioning variables**

```
this.centerX = width / 2;
this.centerY = height / 2;
```

4. **Use percentage CSS (automatic via base class)**
   - The base class already sets `width: 100%` and `height: 100%`
   - You don't need to set this yourself

❌ DON'T:

1. **Don't forget to recreate elements**

```
// ❌ BAD: Only updating variables, elements at old positions
onResize(width, height) {
    this.width = width;
    this.height = height;
}
```

2. **Don’t reset game state**

```
// ❌ BAD: Resets score/wave/lives
onResize(width, height) {
    this.createAliens();  // Calls constructor, resets state
}
```

3. **Don’t manually resize canvas buffer**

```
// ❌ BAD: Base class already does this
onResize(width, height) {
    this.canvas.width = width;
    this.canvas.height = height;
}
```

4. **Don’t use fixed pixel CSS**

```
// ❌ BAD: Doesn't scale with window
this.canvas.style.width = '800px';
```

Quick Reference: Resize Pattern

```
class MyPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myplugin', visualizer, { /* ... */ });

        // Initialize game state variables
        this.score = 0;
        this.lives = 3;
        this.level = 1;

        // Initialize positioning variables
        this.centerX = 0;
        this.centerY = 0;
    }

    onInitialize() {
        // Create initial game objects
        this.createPlayer();
        this.createEnemies();
        this.createBackground();
    }

    onResize(width, height) {
        // 1. Update positioning
        this.centerX = width / 2;
        this.centerY = height / 2;
        this.player.x = width / 2;
        this.player.y = height / 2;

        // 2. Recreate size-dependent elements (with state preservation)
        this.recreateEnemies();
        this.recreateObstacles();
        this.recreateBackground();

        // 3. Update boundaries
        this.leftBoundary = 50;
```

```
        this.rightBoundary = width - 50;
        this.topBoundary = 50;
        this.bottomBoundary = height - 50;
    }

    recreateEnemies() {
        // Helper method that preserves game state
        const currentScore = this.score;
        const currentLevel = this.level;
        const currentLives = this.lives;

        // Recreate enemies
        this.createEnemies();

        // Restore state
        this.score = currentScore;
        this.level = currentLevel;
        this.lives = currentLives;
    }
}
```

2D Canvas Plugins

Basic 2D Plugin Template

No changes required from previous versions - works exactly as before:

```
class My2DPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('my2dplugin', visualizer, {
            version: '1.0.0',
            author: 'Your Name',
            description: '2D Visualization',
            targetFPS: 60
        });

        this.setupControls();
    }

    onInitialize() {
        // Nothing special needed - just use this.ctx
        console.log('2D plugin initialized');
    }

    onRender(deltaTime, timestamp, sharedAudioData) {
        // this.ctx auto-creates 2D context on first access
        this.ctx.fillStyle = 'rgba(0, 0, 0, 0.1)';
        this.ctx.fillRect(0, 0, this.canvas.width, this.canvas.height);

        // Draw visualization
        const energy = this.getAudioEnergy();
        this.ctx.fillStyle = `hsl(${energy * 360}, 100%, 50%)`;
        this.ctx.fillRect(
            this.canvas.width / 2 - 50,
            this.canvas.height / 2 - 50,
            100,
            100
        );
    }
}
```

Key Point: Accessing `this.ctx` automatically creates the 2D context on first use. This is completely backwards compatible with all existing plugins.

WebGL/WebGL2 Plugins

WebGL2 Plugin Template

NEW in v2.2 - Now fully supported:

```
class MyWebGL2Plugin extends FrequePluginBase {
    constructor(visualizer) {
        super('mywebgl2plugin', visualizer, {
            version: '1.0.0',
            author: 'Your Name',
            description: 'WebGL2 Shader Visualization',
            targetFPS: 60
        });

        this.setupControls();
    }

    onInitialize() {
        // Create WebGL2 context BEFORE accessing this.ctx
        this.gl = this.canvas.getContext('webgl2', {
            alpha: false,
            antialias: true,
            powerPreference: 'high-performance',
            preserveDrawingBuffer: false
        });

        if (!this.gl) {
            console.error('WebGL2 not supported');
            return;
        }

        console.log('WebGL2 context created successfully');

        // Initialize shaders, buffers, uniforms
        this.initShaders();
        this.initBuffers();

        // Handle resize
        window.addEventListener('resize', () => this.resize());
        this.resize();
```

```
    }

    initShaders() {
        const gl = this.gl;

        // Vertex shader
        const vertexShaderSource = `#version 300 es
            in vec2 position;
            void main() {
                gl_Position = vec4(position, 0.0, 1.0);
            }
        `;

        // Fragment shader
        const fragmentShaderSource = `#version 300 es
            precision highp float;

            uniform vec2 resolution;
            uniform float time;

            out vec4 fragColor;

            void main() {
                vec2 uv = gl_FragCoord.xy / resolution;
                vec3 color = vec3(uv.x, uv.y, abs(sin(time)));
                fragColor = vec4(color, 1.0);
            }
        `;

        // Compile and link shaders
        const vs = this.compileShader(gl.VERTEX_SHADER, vertexShaderSource);
        const fs = this.compileShader(gl.FRAGMENT_SHADER, fragmentShaderSource);

        this.program = gl.createProgram();
        gl.attachShader(this.program, vs);
        gl.attachShader(this.program, fs);
```

```
        gl.linkProgram(this.program);

        if (!gl.getProgramParameter(this.program, gl.LINK_STATUS)) {
            console.error('Program link error:', gl.getProgramInfoLog(this.program));
            return;
        }

        // Get uniform locations
        this.uniforms = {
            resolution: gl.getUniformLocation(this.program, 'resolution'),
            time: gl.getUniformLocation(this.program, 'time')
        };
    }

    compileShader(type, source) {
        const gl = this.gl;
        const shader = gl.createShader(type);
        gl.shaderSource(shader, source);
        gl.compileShader(shader);

        if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
            console.error('Shader compile error:', gl.getShaderInfoLog(shader));
            gl.deleteShader(shader);
            return null;
        }

        return shader;
    }

    initBuffers() {
        const gl = this.gl;

        // Fullscreen quad
        const vertices = new Float32Array([
            -1, -1,
             1, -1,
```

```
            -1,  1,
             1,  1
        ]);

        const buffer = gl.createBuffer();
        gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
        gl.bufferData(gl.ARRAY_BUFFER, vertices, gl.STATIC_DRAW);

        const positionLoc = gl.getAttribLocation(this.program, 'position');
        gl.enableVertexAttribArray(positionLoc);
        gl.vertexAttribPointer(positionLoc, 2, gl.FLOAT, false, 0, 0);
    }

    resize() {
        if (!this.gl) return;

        // Get dimensions from container, NOT canvas.clientWidth
        const container = document.getElementById('visualizationContainer');
        if (!container) return;

        const rect = container.getBoundingClientRect();
        const width = rect.width || window.innerWidth;
        const height = rect.height || window.innerHeight;

        // Set CSS size
        this.canvas.style.width = width + 'px';
        this.canvas.style.height = height + 'px';

        // Set buffer size with DPI
        const dpr = window.devicePixelRatio || 1;
        this.canvas.width = width * dpr;
        this.canvas.height = height * dpr;

        // Update viewport
        this.gl.viewport(0, 0, this.canvas.width, this.canvas.height);
    }
```

```
    onUpdate(deltaTime, timestamp, sharedAudioData) {
        // Update time for shader animation
        this.time = timestamp * 0.001;
    }

    onRender(deltaTime, timestamp, sharedAudioData) {
        const gl = this.gl;

        if (!gl || !this.program) return;

        gl.useProgram(this.program);

        // Set uniforms
        gl.uniform2f(this.uniforms.resolution, this.canvas.width, this.canvas.height);
        gl.uniform1f(this.uniforms.time, this.time || 0);

        // Draw fullscreen quad
        gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
    }

    shouldClearCanvas() {
        // WebGL handles its own clearing
        return false;
    }
}

// Auto-register
setTimeout(() => {
    if (window.visualizer && window.FrequePluginBase) {
        new MyWebGL2Plugin(window.visualizer);
    }
}, 500);
```

WebGL Plugin Template

For WebGL 1.0 (better compatibility):

```
class MyWebGLPlugin extends FrequePluginBase {
    onInitialize() {
        // Create WebGL context (not WebGL2)
        this.gl = this.canvas.getContext('webgl', {
            alpha: true,
            antialias: true,
            premultipliedAlpha: false
        });

        if (!this.gl) {
            console.error('WebGL not supported');
            return;
        }

        // Use GLSL 1.0 shaders
        const vertexShader = `
            attribute vec2 position;
            void main() {
                gl_Position = vec4(position, 0.0, 1.0);
            }
        `;

        const fragmentShader = `
            precision mediump float;
            uniform vec2 resolution;
            uniform float time;

            void main() {
                vec2 uv = gl_FragCoord.xy / resolution;
                gl_FragColor = vec4(uv.x, uv.y, abs(sin(time)), 1.0);
            }
        `;

        // Setup shaders and buffers...
    }
```

```
    shouldClearCanvas() {
        return false;
    }
}
```

Key Rules for WebGL/WebGL2 Plugins

âœ… DO:

1. **Create context in `onInitialize()`**

   ```
   this.gl = this.canvas.getContext('webgl2', {...});
   ```

2. **Create context BEFORE accessing `this.ctx`**
   - Once you access `this.ctx`, a 2D context is created
   - You cannot get WebGL context after that

3. **Check for context support**

   ```
   if (!this.gl) {
       console.error('WebGL2 not supported');
       return;
   }
   ```

4. **Override `shouldClearCanvas()`**

   ```
   shouldClearCanvas() {
       return false; // WebGL handles its own clearing
   }
   ```

5. **Handle resize manually with container dimensions**

```
resize() {
    if (!this.gl) return;

    // CRITICAL: Get dimensions from container, NOT canvas.clientWidth
    // canvas.clientWidth may return wrong/0 values
    const container = document.getElementById('visualizationContainer');
    if (!container) return;

    const rect = container.getBoundingClientRect();
    const width = rect.width || window.innerWidth;
    const height = rect.height || window.innerHeight;

    // Set CSS display size
    this.canvas.style.width = width + 'px';
    this.canvas.style.height = height + 'px';

    // Set actual buffer size (accounts for DPI)
    const dpr = window.devicePixelRatio || 1;
    this.canvas.width = width * dpr;
    this.canvas.height = height * dpr;

    // Update WebGL viewport
    this.gl.viewport(0, 0, this.canvas.width, this.canvas.height);
}
```

Why not `canvas.clientWidth`?

- `canvas.clientWidth` returns CSS dimensions which may be 0 or incorrect
- Multiplying by `devicePixelRatio` can result in 4x oversized canvas
- Always get dimensions from `visualizationContainer` instead

âŒ DON’T:

1. **Don’t access `this.ctx` in WebGL plugins**
    - Creates a 2D context, breaking WebGL
2. **Don’t rely on base class clearing**
    - WebGL needs `gl.clear()`, not `clearRect()`
3. **Don’t use `canvas.clientWidth` for sizing**
    - Returns CSS dimensions which may be wrong/0
    - Causes 4x oversized canvas bug when multiplied by devicePixelRatio
    - **Always** use `visualizationContainer` dimensions instead
4. **Don’t forget DPI scaling**
    - WebGL canvas needs proper resolution setup
5. **Don’t forget to check browser support**
    - WebGL2 is ~95% supported, but not universal

WebGL vs WebGL2 Comparison

Feature	WebGL 1.0	WebGL2
Browser Support	~97%	~95%
GLSL Version	1.0	3.0 ES
Texture Arrays	âŒ No	âœ… Yes
3D Textures	âŒ No	âœ… Yes
Transform Feedback	âŒ No	âœ… Yes
Multiple Render Targets	âŒ Extension	âœ… Built-in
Integer Textures	âŒ No	âœ… Yes
Sampler Objects	âŒ No	âœ… Yes

Recommendation: - Use **WebGL2** for new advanced shader plugins - Use **WebGL 1.0** for maximum compatibility - Use **Three.js** for 3D scenes (handles both automatically)

Quick Start

Three.js Plugins and Context Handling

IMPORTANT: Three.js plugins work differently from raw WebGL plugins regarding canvas context:

Three.js Handles Context Automatically

```
class MyThreePlugin extends FrequePluginBase {
    onInitialize() {
        // Three.js creates its own WebGL context internally
        this.renderer = new THREE.WebGLRenderer({
            canvas: this.canvas, // Pass this.canvas directly
            alpha: true,
            antialias: true
        });

        // DO NOT call this.canvas.getContext()
        // DO NOT access this.ctx
        // Three.js manages everything
    }
}
```

Why Three.js is Different: - Three.js's `WebGLRenderer` handles context creation internally - You pass `this.canvas` to the renderer, and it does the rest - No need to manually call `getContext()` - Works seamlessly with the new v2.2 architecture

Key Point: Three.js plugins are **unchanged** from previous versions. The new WebGL/WebGL2 architecture only affects plugins that manually create WebGL contexts.

Standard Three.js Setup

Step 1: Get Container Dimensions

CRITICAL: Always get size from `visualizationContainer`, NOT canvas buffer.

```
onInitialize() {
    // Get dimensions from container
    const container = document.getElementById('visualizationContainer');
    const rect = container.getBoundingClientRect();
    const width = rect.width || window.innerWidth;
    const height = rect.height || window.innerHeight;

    console.log(`Container dimensions: ${width}x${height}`);
}
```

Step 2: Create Scene and Camera

```
// Create scene
this.scene = new THREE.Scene();

// Create camera with container aspect ratio
this.camera = new THREE.PerspectiveCamera(
    75,
    width / height,
    0.1,
    1000
);
this.camera.position.z = 20;
this.camera.position.y = 20 * 0.2;  // Vertical offset for better viewing angle
this.camera.lookAt(0, 0, 0);
```

Camera Positioning Note: The Y position is set to 20% of the camera distance (`distance * 0.2`). This provides a slight downward viewing angle that improves depth perception and overall visual quality.

Step 3: Create Renderer

```
this.renderer = new THREE.WebGLRenderer({
    alpha: true,
    antialias: true,
    preserveDrawingBuffer: true
});

// Set pixel ratio for high-DPI displays
this.renderer.setPixelRatio(window.devicePixelRatio);

// Use container dimensions
this.renderer.setSize(width, height);

// Set color space (important for matching Freque's visuals)
if (this.renderer.outputEncoding !== undefined) {
    this.renderer.outputEncoding = THREE.sRGBEncoding;
}
```

Step 4: Setup Canvas CSS

CRITICAL: Set CSS properties individually, NOT with cssText.

```
const threeCanvas = this.renderer.domElement;
threeCanvas.id = this.canvasId;
threeCanvas.className = this.canvas.className;

// Set each CSS property individually
threeCanvas.style.position = 'absolute';
threeCanvas.style.top = '0';
threeCanvas.style.left = '0';
threeCanvas.style.width = '100%';    // Percentage, not pixels
threeCanvas.style.height = '100%';   // Percentage, not pixels
threeCanvas.style.pointerEvents = 'none';
threeCanvas.style.zIndex = this.canvas.style.zIndex || this.zIndex.toString();
threeCanvas.style.display = this.canvas.style.display || 'block';
threeCanvas.style.opacity = this.canvas.style.opacity || '1';
```

Step 5: Replace Canvas in DOM

```
if (this.canvas.parentNode) {
    this.canvas.parentNode.replaceChild(threeCanvas, this.canvas);
}
this.canvas = threeCanvas;
```

Step 6: Add Lighting

```
// Ambient light for overall illumination
const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
this.scene.add(ambientLight);

// Directional lights for reflections
const light1 = new THREE.DirectionalLight(0xffffff, 1.0);
light1.position.set(10, 10, 10);
this.scene.add(light1);

const light2 = new THREE.DirectionalLight(0xffffff, 0.6);
light2.position.set(-10, -10, -10);
this.scene.add(light2);
```

Advanced Three.js Techniques

Video/Background Textures

Finding Video Element

```
// Try multiple selectors
const videoElement = document.querySelector('#bgVideo') ||
                     document.querySelector('#bgVideoPlaceholder') ||
                     document.querySelector('video');

if (!videoElement) {
    console.warn('No video element found');
    return;
}

this.videoElement = videoElement;
```

Finding Background Image

```
const bgImageDiv = document.querySelector('#bgImage');

if (!bgImageDiv) {
    console.warn('No background image div found');
    return;
}

const computedStyle = window.getComputedStyle(bgImageDiv);
const backgroundImage = computedStyle.backgroundImage;

if (!backgroundImage || backgroundImage === 'none') {
    return;
}

// Extract URL from CSS
const urlMatch = backgroundImage.match(/url\(['"]?([^'"]+)['"]?\)/);
if (urlMatch) {
    const imageUrl = urlMatch[1];

    // Load image
    const img = new Image();
    img.crossOrigin = 'anonymous';
    img.src = imageUrl;
    img.onload = () => {
        this.createBackgroundTexture(img);
    };
}
```

Creating Video Texture (Direct Mode)

For mapping video directly onto surfaces:

```
this.videoTexture = new THREE.VideoTexture(videoElement);
this.videoTexture.minFilter = THREE.LinearFilter;
this.videoTexture.magFilter = THREE.LinearFilter;
this.videoTexture.mapping = THREE.UVMapping;  // Standard UV mapping
this.videoTexture.format = THREE.RGBFormat;
this.videoTexture.flipY = true;  // Correct orientation

// Apply to material
material.map = this.videoTexture;
```

Creating Background Image Texture (Direct Mode)

```
this.bgImageTexture = new THREE.Texture(imageElement);
this.bgImageTexture.needsUpdate = true;
this.bgImageTexture.mapping = THREE.UVMapping;
this.bgImageTexture.flipY = true;  // Correct orientation

// Apply to material
material.map = this.bgImageTexture;
```

Cube Camera System for Video Reflections

Why Cube Cameras?

Three.js r120+ doesn’t support video textures with EquirectangularReflectionMapping for environment maps. To get video reflections on chrome/metallic surfaces, you must use a cube camera system.

When to Use Cube Cameras

✅ **Use Cube Cameras For:** - Video reflections on metallic/chrome materials - Dynamic environment mapping - Real-time reflections that change based on object position - Complex reflective surfaces with videos

❌ **Don't Need Cube Cameras For:** - Static background image reflections (use EquirectangularReflectionMapping) - Direct video textures on surfaces (use UVMapping) - Non-reflective materials

Complete Cube Camera Implementation

Step 1: Create Cube Render Target

```
// High resolution for quality (4096 = 4k per cube face)
// Can reduce to 2048 or 1024 for better performance
this.cubeRenderTarget = new THREE.WebGLCubeRenderTarget(4096, {
    format: THREE.RGBAFormat,
    generateMipmaps: true,
    minFilter: THREE.LinearMipmapLinearFilter,
    magFilter: THREE.LinearFilter
});

// Set color space to match renderer
if (this.cubeRenderTarget.texture.encoding !== undefined) {
    this.cubeRenderTarget.texture.encoding = THREE.sRGBEncoding;
}
```

Step 2: Create Cube Camera

```
// Cube camera positioned at origin
this.cubeCamera = new THREE.CubeCamera(
    0.1,   // near
    1000,  // far
    this.cubeRenderTarget
);

this.cubeCamera.position.set(0, 0, 0);

// Enable layer 1 so it can see the video sphere
this.cubeCamera.layers.enable(1);

this.scene.add(this.cubeCamera);

// Store the generated cube map texture
this.cubeMapTexture = this.cubeRenderTarget.texture;
```

Step 3: Create Large Video Sphere

The cube camera needs something to capture. Create a large inverted sphere textured with video:

```
// Large sphere geometry (500 units radius)
const sphereGeometry = new THREE.SphereGeometry(500, 120, 80);
// Invert sphere to face inward
sphereGeometry.scale(-1, 1, 1);

// Create video texture for sphere
const videoTextureForSphere = new THREE.VideoTexture(videoElement);
videoTextureForSphere.minFilter = THREE.LinearFilter;
videoTextureForSphere.magFilter = THREE.LinearFilter;
videoTextureForSphere.mapping = THREE.EquirectangularReflectionMapping;
videoTextureForSphere.format = THREE.RGBAFormat;

// Set color space
if (videoTextureForSphere.encoding !== undefined) {
    videoTextureForSphere.encoding = THREE.sRGBEncoding;
}

// Create material
const sphereMaterial = new THREE.MeshBasicMaterial({
    map: videoTextureForSphere
});

// Create mesh
this.videoSphere = new THREE.Mesh(sphereGeometry, sphereMaterial);

// Base rotations for correct orientation
this.videoSphere.rotation.y = Math.PI / 2;        // 90 degrees
this.videoSphere.rotation.x = -35 * (Math.PI / 180);  // -35 degrees

// Set to layer 1 (hidden from main camera, visible to cube camera)
this.videoSphere.layers.set(1);
this.videoSphere.visible = true;

this.scene.add(this.videoSphere);
```

Step 4: Apply Cube Map to Materials

```
// For chrome/metallic materials
material.map = null;  // Remove direct texture
material.envMap = this.cubeMapTexture;  // Apply cube map
material.metalness = 1.0;
material.roughness = 0.0;
material.envMapIntensity = 0.5;  // Adjust for desired reflection strength
material.needsUpdate = true;
```

Step 5: Update Cube Camera Every Frame

In your `onRender()` method:

```
onRender(deltaTime, timestamp, sharedAudioData) {
    // Update video texture
    if (this.videoElement && !this.videoElement.paused &&
        this.videoElement.readyState >= 2) {

        // Update video texture on sphere
        if (this.videoSphere.material && this.videoSphere.material.map) {
            this.videoSphere.material.map.needsUpdate = true;
        }

        // Update cube camera (required for video reflections)
        // Hide reflective objects temporarily
        this.spheres.forEach(sphere => {
            sphere.mesh.visible = false;
        });

        // Make sure video sphere is visible
        this.videoSphere.visible = true;

        // Update cube camera from origin
        this.cubeCamera.update(this.renderer, this.scene);

        // Restore visibility
        this.spheres.forEach(sphere => {
            sphere.mesh.visible = true;
        });
    }

    // Render main scene
    this.renderer.render(this.scene, this.camera);
}
```

Advanced: Dynamic Cube Camera Positioning

For more realistic reflections that change based on object position:

```
// Calculate average position of all reflective objects
let avgX = 0, avgY = 0, avgZ = 0;
let count = 0;

this.spheres.forEach(sphere => {
    avgX += sphere.mesh.position.x;
    avgY += sphere.mesh.position.y;
    avgZ += sphere.mesh.position.z;
    count++;
});

if (count > 0) {
    avgX /= count;
    avgY /= count;
    avgZ /= count;

    // Move cube camera to average position
    this.cubeCamera.position.set(avgX, avgY, avgZ);
}

// Update cube camera from new position
this.cubeCamera.update(this.renderer, this.scene);
```

Performance Optimization: Higher Quality Captures

```
// Temporarily increase renderer pixel ratio for cube camera
const originalPixelRatio = this.renderer.getPixelRatio();
this.renderer.setPixelRatio(Math.min(originalPixelRatio * 2.0, 2.0));

// Update cube camera
this.cubeCamera.update(this.renderer, this.scene);

// Restore original pixel ratio
this.renderer.setPixelRatio(originalPixelRatio);
```

Anisotropic Filtering for Quality

```
// Add anisotropic filtering to cube map
const maxAnisotropy = this.renderer.capabilities.getMaxAnisotropy();
if (maxAnisotropy > 0) {
    this.cubeMapTexture.anisotropy = maxAnisotropy;
}
```

Automatic Intensity Adjustment for Video Cube Maps

Video cube maps tend to appear brighter than background image reflections due to the rendering pipeline. Chrome Spheres automatically compensates:

```
// Automatic brightness adjustment in updateSphereMaterials()
if (this.mappingMode === 'reflection' && this.envMapSource === 'video') {
    material.envMapIntensity = this.envMapIntensity * 0.8;  // 80% for video
} else {
    material.envMapIntensity = this.envMapIntensity;  // 100% for images
}
```

Why This Matters: - Maintains consistent appearance between video and image sources - User sets one intensity value, system adjusts automatically - Prevents video reflections from being overly bright/washed out - Transparent to users (no extra controls needed) - Improves perceived quality of reflections

Implementation Note: This adjustment happens in `updateSphereMaterials()` and applies every time materials are updated. The 0.8 multiplier was determined through testing to match the visual brightness of background image reflections.

When to Use: Apply source-dependent adjustments when different texture types render with different characteristics:

```
// Generic pattern for source-dependent adjustments
updateMaterials() {
    this.objects.forEach(obj => {
        if (this.sourceType === 'typeA') {
            obj.material.property = this.value * adjustmentFactor;
        } else {
            obj.material.property = this.value;
        }
    });
}
```

Layer Management

Use Three.js layers to control which objects are visible to different cameras.

Layer Setup

```
// Layer 0: Default layer (main camera sees this)
// Layer 1: Hidden from main camera (cube camera sees this)

// Set object to layer 1 (hidden from main camera)
this.videoSphere.layers.set(1);

// Enable cube camera to see layer 1 (while keeping layer 0)
this.cubeCamera.layers.enable(1);
```

Toggling Visibility

```
toggleBackgroundSphere() {
    this.bgSphereVisible = !this.bgSphereVisible;

    // Change layer based on visibility
    this.videoSphere.layers.set(this.bgSphereVisible ? 0 : 1);

    // Additional rotation when visible to user
    if (this.bgSphereVisible) {
        // Add 180° so video appears correct orientation to viewer
        this.videoSphere.rotation.y = this.videoSphereBaseRotationY + Math.PI;
    } else {
        // Use base rotation for cube camera capture
        this.videoSphere.rotation.y = this.videoSphereBaseRotationY;
    }
}
```

Color Space Management

Why sRGB Encoding Matters

To ensure video reflections match the brightness and color of background image reflections:

```
// 1. Set renderer output encoding
this.renderer.outputEncoding = THREE.sRGBEncoding;

// 2. Set cube render target encoding
this.cubeRenderTarget.texture.encoding = THREE.sRGBEncoding;

// 3. Set video texture encoding
videoTexture.encoding = THREE.sRGBEncoding;
```

Result: Consistent color and brightness across all reflection sources.

Dual Mapping Modes

Support both direct texture and reflection modes:

Direct Mode

```
if (this.mappingMode === 'direct') {
    material.map = this.videoTexture;      // Show video directly
    material.envMap = null;                // No environment map
    material.metalness = Math.min(this.metalness, 0.5);
    material.roughness = Math.max(this.roughness, 0.3);
}
```

Reflection Mode

```
if (this.mappingMode === 'reflection') {
    material.map = null;                    // No direct texture
    material.envMap = this.cubeMapTexture; // Use cube map for reflections
    material.metalness = this.metalness;
    material.roughness = this.roughness;
    material.envMapIntensity = this.envMapIntensity;
}
```

Audio Integration

Understanding Audio Data Access (v2.5)

CRITICAL: Two Ways to Access Audio Data

Freque provides audio data through both **base class methods** and **sharedAudioData properties**. Understanding which to use is essential.

✅ CORRECT: Recommended Approach

```
onUpdate(deltaTime, timestamp, sharedAudioData) {
    // ✅ For overall energy - use base class method
    this.audioLevel = this.getAudioEnergy();

    // ✅ For frequency bands - use sharedAudioData properties
    this.bassLevel = sharedAudioData.bass || 0;
    this.midLevel = sharedAudioData.mid || 0;
    this.trebleLevel = sharedAudioData.treble || 0;
    this.beatDetected = sharedAudioData.beat || false;

    // Now use these values in your visualization
    this.updateVisualization();
}
```

Available Audio Sources

From Base Class Methods: - `this.getAudioEnergy()` - Returns overall energy level (0.0-1.0) - `this.getAudioFrequencies()` - Returns raw frequency data array (Uint8Array)

From sharedAudioData Parameter: - `sharedAudioData.bass` - Bass level (0.0-1.0) - `sharedAudioData.mid` - Mid-range level (0.0-1.0) - `sharedAudioData.treble` - Treble level (0.0-1.0) - `sharedAudioData.beat` - Beat detected (boolean)

Complete Audio Integration Example

```
class MyAudioPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myaudioplugin', visualizer, {
            version: '1.0.0',
            author: 'Your Name',
            description: 'Audio-reactive visualization'
        });

        // Initialize audio variables in constructor
        this.audioLevel = 0;
        this.bassLevel = 0;
        this.midLevel = 0;
        this.trebleLevel = 0;
        this.beatDetected = false;

        this.setupControls();
    }

    onUpdate(deltaTime, timestamp, sharedAudioData) {
        // Get audio data
        this.audioLevel = this.getAudioEnergy();
        this.bassLevel = sharedAudioData.bass || 0;
        this.midLevel = sharedAudioData.mid || 0;
        this.trebleLevel = sharedAudioData.treble || 0;
        this.beatDetected = sharedAudioData.beat || false;

        // React to audio
        this.size = 100 + (this.bassLevel * 200);        // Bass affects size
        this.rotation += this.midLevel * 0.1;             // Mid affects rotation
        this.brightness = 0.5 + (this.trebleLevel * 0.5); // Treble affects brightness

        if (this.beatDetected) {
            this.triggerPulse();
        }
    }
```

```
    triggerPulse() {
        // Special effect on beat
        this.pulseScale = 1.5;
    }

    onRender(deltaTime, timestamp, sharedAudioData) {
        // Use audio-reactive variables in rendering
        this.ctx.fillStyle = `hsl(${this.midLevel * 360}, 100%, ${this.brightness * 100}%
        // ... render visualization
    }
}
```

Audio Reactivity Mapping Patterns

Pattern 1: Bass → Size/Scale

```
const scale = 1.0 + (this.bassLevel * 2.0);
sphere.scale.set(scale, scale, scale);
```

Pattern 2: Mid → Movement/Speed

```
this.velocity = this.midLevel * 5.0;
this.position += this.velocity * deltaTime;
```

Pattern 3: Treble → Brightness/Detail

```
this.brightness = 0.5 + (this.trebleLevel * 0.5);
material.emissiveIntensity = this.trebleLevel;
```

Pattern 4: Beat → Trigger Events

```
if (this.beatDetected) {
    this.createExplosion();
    this.flashScreen();
}
```

Pattern 5: Overall Energy → Global Intensity

```
const energy = this.getAudioEnergy();
this.globalIntensity = energy;
this.particleCount = Math.floor(10 + energy * 100);
```

Getting Audio Data

```
onUpdate(deltaTime, timestamp, sharedAudioData) {
    // Get overall energy (0-1)
    const energy = this.getAudioEnergy();

    // Get frequency data
    const frequencies = this.getAudioFrequencies();

    // Get frequency bands
    this.frequencyBands = this.getFrequencyBands();

    // Detect beats
    const isBeat = this.detectBeat(timestamp);
    if (isBeat) {
        this.onBeat();
    }
}
```

Frequency Bands

```
getFrequencyBands() {
    const frequencies = this.getAudioFrequencies();

    if (!frequencies || frequencies.length === 0) {
        return { bass: 0, mid: 0, treble: 0 };
    }

    const length = frequencies.length;

    return {
        bass: this.avgRange(frequencies, 0, Math.floor(length * 0.1)),
        mid: this.avgRange(frequencies, Math.floor(length * 0.1), Math.floor(length * 0.5
        treble: this.avgRange(frequencies, Math.floor(length * 0.5), length)
    };
}

avgRange(arr, start, end) {
    if (!arr || arr.length === 0) return 0;

    let sum = 0;
    let count = 0;

    for (let i = start; i < end && i < arr.length; i++) {
        sum += arr[i] / 255;  // Normalize to 0-1
        count++;
    }

    return count > 0 ? sum / count : 0;
}
```

Beat Detection

```
// In constructor
this.currentEnergy = 0;
this.smoothedEnergy = 0;
this.energyHistory = [];
this.beatDetection = {
    lastBeatTime: 0,
    threshold: 1.3,
    minTimeBetweenBeats: 200
};

// In onUpdate
onUpdate(deltaTime, timestamp, sharedAudioData) {
    this.currentEnergy = this.getAudioEnergy();

    // Smooth energy
    this.energyHistory.push(this.currentEnergy);
    if (this.energyHistory.length > 30) {
        this.energyHistory.shift();
    }

    this.smoothedEnergy = this.energyHistory.reduce((a, b) => a + b, 0) / this.energyHist

    // Detect beat
    const isBeat = this.detectBeat(timestamp);
    if (isBeat && this.beatPulse) {
        this.onBeat();
    }
}

detectBeat(timestamp) {
    const energy = this.currentEnergy;
    const bd = this.beatDetection;
    const avg = this.smoothedEnergy;

    const timeSinceBeat = timestamp - bd.lastBeatTime;
```

```
    if (energy > avg * bd.threshold && timeSinceBeat > bd.minTimeBetweenBeats) {
        bd.lastBeatTime = timestamp;
        return true;
    }

    return false;
}

detectBeat(timestamp) {
    const energy = this.currentEnergy;
    const bd = this.beatDetection;
    const avg = this.smoothedEnergy;

    const timeSinceBeat = timestamp - bd.lastBeatTime;
    if (energy > avg * bd.threshold && timeSinceBeat > bd.minTimeBetweenBeats) {
        bd.lastBeatTime = timestamp;
        return true;
    }

    return false;
}
```

Energy Smoothing Configuration: The energy history uses a **10-sample rolling average** for responsive but smooth beat detection:

```
// In onUpdate()
this.energyHistory.push(this.currentEnergy);
if (this.energyHistory.length > 10) {  // Buffer size: 10 samples
    this.energyHistory.shift();
}
this.smoothedEnergy = this.energyHistory.reduce((a, b) => a + b, 0) / this.energyHistory.
```

Why 10 Samples? - Too few (3-5): Overly sensitive, false positives - Just right (10): Smooth but responsive - Too many (20+): Sluggish response, missed beats

Tuning Beat Detection: - `threshold`: 1.3 = energy must be 30% above average - `minTimeBetweenBeats`: 200ms prevents double-triggers - Lower threshold = more sensitive - Higher threshold = only strong beats

onBeat() { // React to beat - e.g., pulse spheres this.spheres.forEach(sphere => { const currentScale = sphere.mesh.scale.x; sphere.mesh.scale.set(currentScale * 1.2, currentScale * 1.2, currentScale * 1.2);

```
    setTimeout(() => {
        sphere.mesh.scale.set(1, 1, 1);
    }, 100);
});
```

}

```


## Audio-Reactive Effects

### Scale with Bass

```javascript
if (this.audioReactive && this.bassScale) {
    const bassScale = 1 + this.frequencyBands.bass * 0.5;
    sphere.mesh.scale.set(bassScale, bassScale, bassScale);
}
```

Movement with Mids

```
if (this.audioReactive && this.midMovement) {
    const movementBoost = 1 + this.frequencyBands.mid * 3;
    sphere.mesh.position.x = sphere.basePosition.x + floatX * movementBoost;
    sphere.mesh.position.y = sphere.basePosition.y + floatY * movementBoost;
}
```
```

Roughness with Treble

```
if (this.audioReactive && this.trebleShine) {
    const dynamicRoughness = this.roughness * (1 - this.frequencyBands.treble * 0.5);
    sphere.mesh.material.roughness = Math.max(0.01, dynamicRoughness);
}
```

Float Speed with Non-Linear Scaling

For controls that need finer precision in certain ranges, use two-part scaling:

```
// Float Speed: 0-10 with custom scaling curve
// Center point at 3 = normal speed (1.0x)
const speedFactor = this.floatSpeed <= 3
    ? this.floatSpeed / 3.0          // 0-3 → 0-1.0x (slow motion to normal)
    : 1.0 + ((this.floatSpeed - 3) / 7.0);  // 3-10 → 1.0-2.0x (normal to fast)

this.time += deltaTime * 0.001 * speedFactor;
```

Why Use Two-Part Scaling? - Provides finer control in the slow-motion range (0-3) - Normal speed at center point (3) for easy reset - Acceleration range (3-10) for dramatic effects - More intuitive than linear 0-2x scaling

User Experience: - Slider starts at 3 (normal speed) - Going below 3 slows down (useful for detailed viewing) - Going above 3 speeds up (useful for energetic music) - 0 = completely frozen - 10 = double speed

Implementation Pattern:

```
// Generic non-linear scaling
const centerValue = 3;
const scaledOutput = controlValue <= centerValue
    ? (controlValue / centerValue)
    : 1.0 + ((controlValue - centerValue) / (maxValue - centerValue));
```

UI Controls

Dropdowns

CRITICAL Requirements for Dropdowns

Dropdowns must have: 1. `type: 'dropdown'` 2. `className: 'dropdown-selector-mixer'` **(REQUIRED for proper display)** 3. `options` array with objects containing `value` and `label` properties 4. Initial `value` that matches one of the option values 5. `onChange` callback

❌ WRONG Dropdown Formats

```
// ❌ WRONG - Simple array (won't work)
options: ['Option 1', 'Option 2', 'Option 3']

// ❌ WRONG - Object with key-value pairs (won't work)
options: {
    'value1': 'Label 1',
    'value2': 'Label 2'
}

// ❌ WRONG - Missing className (dropdown won't render items)
this.addControl('myDropdown', {
    type: 'dropdown',
    options: [{ value: 0, label: 'Option' }]
    // Missing className!
});
```

✅ CORRECT Dropdown Format

```
this.addControl('colorScheme', {
    type: 'dropdown',
    label: 'Color Scheme',
    className: 'dropdown-selector-mixer',  // ✅ REQUIRED
    options: [
        { value: 'classic', label: 'Classic' },
        { value: 'rainbow', label: 'Rainbow' },
        { value: 'neon', label: 'Neon' },
        { value: 'retro', label: 'Retro' }
    ],
    value: 'classic',  // Must match one of the values above
    onChange: (value) => {
        this.colorScheme = value;  // value will be 'classic', 'rainbow', etc.
    }
});
```

Dropdown Value Types

String Values (Recommended for named options):

```
// ✅ Best for mode selection
this.addControl('mode', {
    type: 'dropdown',
    label: 'Mode',
    className: 'dropdown-selector-mixer',
    options: [
        { value: 'slow', label: 'Slow' },
        { value: 'medium', label: 'Medium' },
        { value: 'fast', label: 'Fast' }
    ],
    value: 'medium',
    onChange: (value) => {
        this.mode = value;  // value is 'slow', 'medium', or 'fast'
    }
});
```

Numeric Values (For indexed options):

```
// ✅ Works for scheme indexes
this.addControl('colorScheme', {
    type: 'dropdown',
    label: 'Color Scheme',
    className: 'dropdown-selector-mixer',
    options: [
        { value: 0, label: 'Acid Orange/Cyan' },
        { value: 1, label: 'Blue Soap Bubbles' },
        { value: 2, label: 'Red Lava' },
        { value: 3, label: 'Purple Dream' },
        { value: 4, label: 'Rainbow' }
    ],
    value: 0,
    onChange: (value) => {
        this.colorScheme = parseInt(value);  // Convert to number
    }
});
```

Float Values (For preset speeds/multipliers):

```
// ✅ Works for speed multipliers
this.addControl('morphSpeed', {
    type: 'dropdown',
    label: 'Morph Speed',
    className: 'dropdown-selector-mixer',
    options: [
        { value: 0.5, label: 'Slow' },
        { value: 1.0, label: 'Medium' },
        { value: 4.0, label: 'Fast' },
        { value: 12.0, label: 'Ultra' }
    ],
    value: 1.0,
    onChange: (value) => {
        this.morphSpeed = parseFloat(value);  // Convert to float
    }
});
```

Complete Dropdown Example with Usage

```
class MyPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myplugin', visualizer, {...});

        // ✅ CRITICAL: Initialize dropdown variable in constructor
        this.colorScheme = 'classic';

        this.setupControls();
    }

    setupControls() {
        this.addControl('colorScheme', {
            type: 'dropdown',
            label: 'Color Scheme',
            className: 'dropdown-selector-mixer',  // REQUIRED
            options: [
                { value: 'classic', label: 'Classic Green' },
                { value: 'rainbow', label: 'Rainbow' },
                { value: 'neon', label: 'Neon Pink' },
                { value: 'retro', label: 'Retro Orange' },
                { value: 'plasma', label: 'Plasma Purple' },
                { value: 'monochrome', label: 'Monochrome' }
            ],
            value: 'classic',  // Must match initialized value
            onChange: (value) => {
                this.colorScheme = value;
                // Colors will update automatically on next render
            }
        });
    }

    // ✅ Use the dropdown value in your rendering
    getColor(index) {
        const schemes = {
            classic: ['#0f0', '#0ff', '#fff'],
```

```
            rainbow: ['#f0f', '#0ff', '#ff0', '#f00', '#0f0'],
            neon: ['#f0f', '#0ff', '#ff0'],
            retro: ['#ff6b35', '#f7931e', '#fdc82f'],
            plasma: ['#ff006e', '#8338ec', '#3a86ff'],
            monochrome: ['#fff', '#ccc', '#999']
        };

        const colors = schemes[this.colorScheme] || schemes.classic;
        return colors[index % colors.length];
    }
}
```

Dropdown Troubleshooting

Problem: Dropdown shows no items - ✅ Check that `className: 'dropdown-selector-mixer'` is present - ✅ Verify options format: `[{ value: 'x', label: 'X' }, ...]` - ✅ Ensure options array is not empty - ✅ Check browser console for errors

Problem: onChange receives undefined - ✅ Check that option `value` properties are set correctly - ✅ Verify initial `value` matches one of the option values exactly - ✅ Make sure you're not using a simple array format

Problem: Selected value doesn't update visualization - ✅ Ensure the variable is initialized in constructor
- ✅ Check that the value is being used in onRender() or onUpdate() - ✅ Verify the mapping logic (like color schemes object) is correct - ✅ Add console.log in onChange to verify it's being called

Dropdown with Object Options

```
// Store schemes as objects
this.colorSchemes = [
    { name: 'Acid Orange/Cyan', id: 0 },
    { name: 'Blue Soap Bubbles', id: 1 },
    { name: 'Red Lava', id: 2 }
];

// Map to options format
this.addControl('colorScheme', {
    type: 'dropdown',
    label: 'Color Scheme',
    className: 'dropdown-selector-mixer',
    options: this.colorSchemes.map(s => ({ value: s.id, label: s.name })),
    value: 0,
    onChange: (value) => {
        this.colorScheme = parseInt(value);
    }
});
```

Why className is Required

Before v2.2 Fix:

```
// ❌ WRONG - dropdown won't display properly
this.addControl('myDropdown', {
    type: 'dropdown',
    options: [...],
    // Missing className causes broken display!
});
```

After v2.2 Fix:

```
// ✅ CORRECT - respects plugin's className
this.addControl('myDropdown', {
    type: 'dropdown',
    className: 'dropdown-selector-mixer',  // Styled correctly
    options: [...]
});
```

What Changed: The `plugin-mixer-integration.js` now respects the plugin's `className` instead of hardcoding `dropdown-mini`. This allows plugins to use the properly-styled `dropdown-selector-mixer` class.

Toggle Controls (Checkboxes)

CRITICAL: Use type: 'checkbox' with className: 'btn-primary-mixer'

Toggle controls must use `type: 'checkbox'` with `className: 'btn-primary-mixer'`, `checked`, and `onChange`:

```
// ✅ CORRECT - Use checkbox type with proper className for toggles
this.colorMorph = false;
this.addControl('colorMorph', {
    type: 'checkbox',
    label: 'Color Morph',
    checked: false,
    className: 'btn-primary-mixer',  // REQUIRED for consistent styling
    onChange: (value) => {
        this.colorMorph = value;
        console.log('Color Morph:', value);
    }
});
```

Toggle That Starts ON

```
// ✅ CORRECT - Starts enabled
this.audioReactive = true;
this.addControl('audioReactive', {
    type: 'checkbox',
    label: 'Audio Reactive',
    checked: true,  // Starts ON
    className: 'btn-primary-mixer',
    onChange: (value) => {
        this.audioReactive = value;
        console.log('Audio Reactive:', value);
    }
});
```

Complete Toggle Example

```
// In constructor:
this.animationMorph = false;

// In setupControls():
this.addControl('animationMorph', {
    type: 'checkbox',
    label: 'Animation Morph',
    checked: false,
    className: 'btn-primary-mixer',
    onChange: (value) => {
        this.animationMorph = value;
        // Value updates automatically - no manual DOM manipulation needed!
    }
});

// In onUpdate() or onRender():
if (this.animationMorph) {
    // Apply animation morph effect
}
```

Why Checkbox Instead of Button?

The Freque control system has two different types:

1. `type: 'checkbox'` - For toggles with ON/OFF states
    - Manages state internally
    - Updates automatically
    - Visual toggle styling
    - Use for: Enable/disable features
2. `type: 'button'` - For one-time actions
    - Triggers onClick callback

- No state management
- Button styling
- Use for: Mode switching, actions

❌ WRONG Pattern (Don't Use Button for Toggles)

```
// ❌ BROKEN - Don't use button type for toggles
this.addControl('myToggle', {
    type: 'button',
    label: 'Toggle: OFF',
    onClick: () => {
        this.toggleMyToggle();  // Manual state management required
    }
});

// ❌ BROKEN - Requires manual DOM manipulation
toggleMyToggle() {
    this.myToggle = !this.myToggle;
    const btn = document.querySelector('[data-control="myToggle"]');
    // Manual text updates, class toggling, etc.
}
```

✅ CORRECT Pattern (Use Checkbox)

```
// ✅ WORKS - Checkbox handles everything automatically
this.addControl('myToggle', {
    type: 'checkbox',
    label: 'My Toggle',
    checked: false,
    onChange: (value) => {
        this.myToggle = value;  // That's it!
    }
});
```

Mode Toggle Buttons (Two-State Switching)

Use `type: 'button'` with `onClick` for switching between two modes (not ON/OFF):

For buttons that toggle between two modes like “Direct” ↔ “Reflection”:

```
this.mappingMode = 'reflection';  // Initial mode

this.addControl('mappingMode', {
    type: 'button',
    label: 'Reflection',
    className: 'btn-toggle active',
    onClick: () => {
        this.toggleMappingMode();
    }
});
```

Updating Mode Button Labels

```
toggleMappingMode() {
    // Switch between two modes
    this.mappingMode = this.mappingMode === 'direct' ? 'reflection' : 'direct';

    // Query DOM directly
    const buttonElement = document.querySelector('[data-control="mappingMode"]');

    if (buttonElement) {
        // Update label to show current mode
        const newLabel = this.mappingMode === 'direct' ? 'Direct Texture' : 'Reflection';
        buttonElement.textContent = newLabel;

        // Toggle active class
        if (this.mappingMode === 'reflection') {
            buttonElement.classList.add('active');
        } else {
            buttonElement.classList.remove('active');
        }
    }

    // Apply the mode change
    this.applyTextureToSpheres();
}
```

When to Use Each Type

Control Type	Use For	Example
`type: 'checkbox'`	ON/OFF toggles	Audio Reactive, Color Morph, Solid Fill
`type: 'button'`	Mode switching	Direct ↔ Reflection, Video ↔ Image
`type: 'button'`	One-time actions	Randomize, Reset, Trigger

Dials (Rotary Controls)

Basic Dial

```
this.addControl('size', {
    type: 'dial',
    label: 'Size',
    min: 5.0,
    max: 10.0,
    step: 0.1,
    value: this.maxSize,
    onChange: (value) => {
        this.maxSize = value;
        this.updateSphereSizes();
    }
});
```

Dial with Units

```
this.addControl('floatSpeed', {
    type: 'dial',
    label: 'Float Speed',
    min: 0,
    max: 10,
    step: 0.1,
    value: this.floatSpeed,
    unit: 'x',  // Shows "5.0x" instead of just "5.0"
    onChange: (value) => {
        this.floatSpeed = value;
    }
});
```

Note: Dials (rotary controls) have replaced sliders in v2.3 for better visual consistency with professional audio/video software.

Custom Dial Fill Colors

Plugins can specify a custom fill color for all their dials using CSS variables:

```
super('myplugin', visualizer, {
    version: '1.0.0',
    author: 'Your Name',
    description: 'My Awesome Plugin',
    dialFillColor: '--accent-color' // All dials will use accent color from theme
});
```

How It Works: - Set `dialFillColor` in plugin config (constructor's second parameter) - Value should be a CSS variable name (e.g., `'--accent-color'`, `'--highlight-color'`, `'--error-color'`) - All dials in the plugin will use this color for their fill - Dial outline (stroke) remains unchanged (uses `--text-secondary`) - If not specified, dials use default `dial-gold` styling - Works with all themes (color changes automatically with theme)

Available CSS Variables: - `--accent-color` - Theme accent color (teal in Light-Vibrant) - `--highlight-color` - Theme highlight color - `--error-color` - Error/warning color - `--success-color` - Success color - `--warning-color` - Warning color - Any other CSS variable defined in your theme

Example:

```
class MyPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myplugin', visualizer, {
            version: '1.0.0',
            dialFillColor: '--accent-color' // All dials will be teal in Light-Vibrant th
        });
    }
}
```

Additional Checkbox Examples

For more checkbox/toggle examples, see the Toggle Controls section above.

Basic checkbox (same as toggle):

```
this.addControl('showGrid', {
    type: 'checkbox',
    label: 'Show Grid',
    checked: false,
    className: 'btn-primary-mixer',
    onChange: (checked) => {
        this.showGrid = checked;
    }
});
```

Control Variable Initialization (v2.5)

CRITICAL: Initialize All Control Variables in Constructor

When controls are created, their `onChange` callbacks don't fire until the user interacts with them. If you don't initialize the variables in the constructor, they'll be `undefined` until the user touches the control.

❌ WRONG - Variables Not Initialized

```
class MyPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myplugin', visualizer, {...});

        // ❌ No initialization - variables are undefined!
        this.setupControls();
    }

    setupControls() {
        this.addControl('laserFreq', {
            type: 'dial',
            min: 0,
            max: 100,
            value: 30,
            onChange: (value) => {
                this.laserFreq = value;  // Only runs when user changes it!
            }
        });
    }

    onUpdate() {
        // ❌ this.laserFreq is undefined until user touches the dial!
        const fireChance = this.laserFreq / 100;  // NaN!
    }
}
```

✅ CORRECT - All Variables Initialized

```
class MyPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myplugin', visualizer, {...});

        // ✅ Initialize ALL control variables with default values
        this.alienSize = 40;
        this.alienCount = 33;
        this.moveSpeed = 1.0;
        this.laserFreq = 30;
        this.colorScheme = 'classic';
        this.displayScore = true;
        this.displayBarriers = true;

        this.setupControls();
    }

    setupControls() {
        this.addControl('laserFreq', {
            type: 'dial',
            min: 0,
            max: 100,
            value: 30,  // Should match the initialized value above
            onChange: (value) => {
                this.laserFreq = value;
            }
        });

        // ... more controls
    }

    onUpdate() {
        // ✅ this.laserFreq is always defined (30 initially)
        const fireChance = this.laserFreq / 100;  // Works immediately!
```

```
    }
}
```

Best Practice Pattern

```
constructor(visualizer) {
    super('myplugin', visualizer, {...});

    // STEP 1: Initialize data structures
    this.particles = [];
    this.effects = [];

    // STEP 2: Initialize control variables with defaults
    // (Match the 'value' you'll use in addControl)
    this.particleCount = 100;
    this.size = 5.0;
    this.speed = 1.0;
    this.colorScheme = 'rainbow';
    this.audioReactive = true;

    // STEP 3: Initialize audio variables
    this.audioLevel = 0;
    this.bassLevel = 0;
    this.midLevel = 0;
    this.trebleLevel = 0;

    // STEP 4: Setup controls and presets
    this.setupControls();
    this.setupPresets();
}
```

Checklist for Every Control

When adding a control, ensure: 1. ✅ Variable is initialized in constructor with default value 2. ✅ Control `value` matches the initialized variable value 3. ✅ `onChange` callback updates the variable 4. ✅ Variable is actually USED in `onUpdate()` or `onRender()` 5. ✅ Changing the control produces a visible effect

---

Control Update Behavior

Understanding when controls take effect is important for user experience:

Immediate Updates

These controls update **instantly** when changed (may cause visual “pop”):

Geometry Changes: - **Sphere Count** → `recreateSpheres()` - Creates/removes spheres immediately - **Max Size** → `updateSphereSizes()` - Updates geometry instantly - **Spread** → `updateSpherePositions()` - Repositions spheres immediately

```
updateSpherePositions() {
    this.spheres.forEach((sphere) => {
        // Calculate new position
        sphere.basePosition.set(newX, newY, newZ);

        // Update mesh position immediately (not deferred)
        sphere.mesh.position.copy(sphere.basePosition);
    });
}
```

Material Changes: - **Metalness** → `updateSphereMaterials()` - Updates material immediately - **Roughness** → `updateSphereMaterials()` - Updates material immediately - **Reflection Intensity** → `updateSphereMaterials()` - Updates immediately

Deferred Updates

These controls affect behavior in next frame (smooth transition):

Animation Properties: - **Float Speed** → Affects `deltaTime` scaling in next `onUpdate()` - **Camera Distance** → Updates camera position next frame - **Camera Auto-Rotate** → Affects rotation in next frame

Audio Toggles: - **Audio Reactive** → Affects processing in next `onUpdate()` - **Bass/Mid/Treble Toggles** → Apply in next audio analysis

Why It Matters

Immediate updates: - ✅ Instant visual feedback - ✅ User knows change happened - ⚠️ May cause brief visual discontinuity

Deferred updates: - ✅ Smooth transitions - ✅ No visual “pop” - ⚠️ Effect not instantly apparent

Best Practice: Use immediate updates for properties where users expect instant feedback (geometry, materials). Use deferred updates for properties affecting motion or animation flow.

Color Morphing Techniques (v2.5)

Color morphing creates smooth, dynamic color transitions that enhance visual appeal and audio reactivity. This section covers implementation patterns for various color morphing techniques.

What is Color Morphing?

Color morphing is the smooth transition between colors over time, creating flowing, organic color changes. Common uses: - Cycling through a color palette - Audio-reactive color shifts - Smooth transitions between user-selected schemes - Dynamic hue/saturation/brightness changes

Basic Color Morph Implementation

HSL Color Space (Recommended)

HSL (Hue, Saturation, Lightness) is ideal for morphing because you can smoothly animate hue while keeping saturation and lightness constant.

```
class MyPlugin extends FrequePluginBase {
    constructor(visualizer) {
        super('myplugin', visualizer, {...});

        // Color morph state
        this.colorMorph = false;
        this.hue = 0;
        this.morphSpeed = 1.0;

        this.setupControls();
    }

    setupControls() {
        // Toggle color morphing
        this.addControl('colorMorph', {
            type: 'checkbox',
            label: 'Color Morph',
            checked: false,
            className: 'btn-primary-mixer',
            onChange: (value) => {
                this.colorMorph = value;
            }
        });

        // Morph speed control
        this.addControl('morphSpeed', {
            type: 'dial',
            label: 'Morph Speed',
            min: 0.1,
            max: 5.0,
            step: 0.1,
            value: 1.0,
            onChange: (value) => {
                this.morphSpeed = value;
            }
```

```
        });
    }

    onUpdate(deltaTime, timestamp) {
        if (this.colorMorph) {
            // Increment hue (0-360 degrees)
            this.hue += this.morphSpeed * deltaTime * 0.05;
            if (this.hue > 360) this.hue -= 360;
        }
    }

    onRender(deltaTime, timestamp, sharedAudioData) {
        // Use morphing hue
        const color = `hsl(${this.hue}, 100%, 50%)`;
        this.ctx.fillStyle = color;
        // ... render visualization
    }
}
```

Advanced Color Morph Patterns

Pattern 1: Palette Cycling

Smoothly cycle through multiple predefined colors:

```
constructor(visualizer) {
    super('myplugin', visualizer, {...});

    this.colorMorph = false;
    this.colorPalette = [
        { h: 0, s: 100, l: 50 },    // Red
        { h: 60, s: 100, l: 50 },   // Yellow
        { h: 120, s: 100, l: 50 },  // Green
        { h: 180, s: 100, l: 50 },  // Cyan
        { h: 240, s: 100, l: 50 },  // Blue
        { h: 300, s: 100, l: 50 }   // Magenta
    ];
    this.paletteIndex = 0;
    this.paletteProgress = 0;
    this.morphSpeed = 1.0;
}

onUpdate(deltaTime, timestamp) {
    if (this.colorMorph) {
        // Progress through palette
        this.paletteProgress += this.morphSpeed * deltaTime * 0.001;

        if (this.paletteProgress >= 1.0) {
            this.paletteProgress = 0;
            this.paletteIndex = (this.paletteIndex + 1) % this.colorPalette.length;
        }
    }
}

getCurrentColor() {
    if (!this.colorMorph) {
        return this.colorPalette[0];
    }

    const currentColor = this.colorPalette[this.paletteIndex];
```

```
    const nextColor = this.colorPalette[(this.paletteIndex + 1) % this.colorPalette.lengt

    // Interpolate between colors
    const h = this.lerp(currentColor.h, nextColor.h, this.paletteProgress);
    const s = this.lerp(currentColor.s, nextColor.s, this.paletteProgress);
    const l = this.lerp(currentColor.l, nextColor.l, this.paletteProgress);

    return { h, s, l };
}

lerp(a, b, t) {
    return a + (b - a) * t;
}

onRender(deltaTime, timestamp, sharedAudioData) {
    const color = this.getCurrentColor();
    this.ctx.fillStyle = `hsl(${color.h}, ${color.s}%, ${color.l}%)`;
    // ... render
}
```

Pattern 2: Audio-Reactive Color Morph

Color morphing that responds to audio:

```
onUpdate(deltaTime, timestamp, sharedAudioData) {
    if (this.colorMorph) {
        // Base morph speed
        let speed = this.morphSpeed * deltaTime * 0.05;

        // Audio modulation
        if (sharedAudioData) {
            const bass = sharedAudioData.bass || 0;
            const mid = sharedAudioData.mid || 0;

            // Bass affects morph speed
            speed *= (1 + bass * 2);

            // Mid affects saturation
            this.saturation = 50 + (mid * 50);
        }

        this.hue += speed;
        if (this.hue > 360) this.hue -= 360;
    }
}

onRender(deltaTime, timestamp, sharedAudioData) {
    const color = `hsl(${this.hue}, ${this.saturation}%, 50%)`;
    this.ctx.fillStyle = color;
    // ... render
}
```

Pattern 3: Per-Element Color Morph

Different elements morph at different speeds/offsets:

```
constructor(visualizer) {
    super('myplugin', visualizer, {...});

    this.particles = [];
    this.colorMorph = false;
    this.baseHue = 0;
}

createParticles() {
    for (let i = 0; i < 100; i++) {
        this.particles.push({
            x: Math.random() * this.canvas.width,
            y: Math.random() * this.canvas.height,
            hueOffset: Math.random() * 360,  // Each particle has unique hue offset
            speed: 0.5 + Math.random() * 1.5
        });
    }
}

onUpdate(deltaTime, timestamp) {
    if (this.colorMorph) {
        this.baseHue += deltaTime * 0.05;
        if (this.baseHue > 360) this.baseHue -= 360;
    }
}

onRender(deltaTime, timestamp, sharedAudioData) {
    this.particles.forEach(particle => {
        // Each particle has unique color based on its offset
        const hue = (this.baseHue + particle.hueOffset * particle.speed) % 360;
        this.ctx.fillStyle = `hsl(${hue}, 100%, 50%)`;
        this.ctx.beginPath();
        this.ctx.arc(particle.x, particle.y, 5, 0, Math.PI * 2);
        this.ctx.fill();
```

```
    });
}
```

Color Morph Best Practices

1. Provide User Control

Always give users control over: - **Toggle** - Enable/disable morphing - **Speed** - How fast colors change

```
this.addControl('colorMorph', {
    type: 'checkbox',
    label: 'Color Morph',
    checked: false,
    className: 'btn-primary-mixer',
    onChange: (value) => { this.colorMorph = value; }
});

this.addControl('morphSpeed', {
    type: 'dial',
    label: 'Morph Speed',
    min: 0,
    max: 5.0,
    step: 0.1,
    value: 1.0,
    onChange: (value) => { this.morphSpeed = value; }
});
```

2. Handle Hue Wrapping

Hue is circular (0-360), so handle wrapping correctly:

```
// ✅ CORRECT - Handle circular interpolation
lerpHue(a, b, t) {
    const diff = b - a;

    // Find shortest path around the circle
    if (diff > 180) {
        b -= 360;
    } else if (diff < -180) {
        b += 360;
    }

    let result = a + (b - a) * t;

    // Normalize to 0-360
    if (result < 0) result += 360;
    if (result > 360) result -= 360;

    return result;
}
```

3. Combine with Static Schemes

Allow users to choose between static colors and morphing:

```
getColor(index) {
    if (this.colorMorph) {
        // Use morphing color
        return `hsl(${this.hue}, 100%, 50%)`;
    } else {
        // Use static scheme
        const colors = this.schemes[this.colorScheme];
        return colors[index % colors.length];
    }
}
```

Advanced Audio Reactivity

Global Audio Controls

For fine-tuned control over how audio affects your visualization, implement global sensitivity and smoothing controls.

Audio Sensitivity

Global multiplier that scales all audio influence (0-200%):

```
// In constructor
this.audioSensitivity = 1.0; // 100% = 1.0, 200% = 2.0

// Add control
this.addControl('audioSensitivity', {
    type: 'dial',
    label: 'Audio Sensitivity',
    min: 0,
    max: 200,
    step: 5,
    value: 100,
    onChange: (value) => {
        this.audioSensitivity = value / 100; // Convert 0-200% to 0-2.0
    }
});

// Apply in getAudioData
getAudioData() {
    // ... get and smooth raw values

    return {
        bass: this.smoothedBass * this.audioSensitivity,
        mid: this.smoothedMid * this.audioSensitivity,
        treble: this.smoothedTreble * this.audioSensitivity,
        energy: this.smoothedEnergy * this.audioSensitivity
    };
}
```

Audio Smoothing

Control the exponential smoothing factor to prevent jittery movement (0-100%):

```
// In constructor
this.audioSmoothing = 0.15; // 15% smoothing
this.audioSmoothingFactor = this.audioSmoothing;

// Add control
this.addControl('audioSmoothing', {
    type: 'dial',
    label: 'Audio Smoothing',
    min: 0,
    max: 100,
    step: 5,
    value: 15,
    onChange: (value) => {
        this.audioSmoothing = value / 100; // Convert 0-100% to 0-1.0
        this.audioSmoothingFactor = this.audioSmoothing;
    }
});

// Use in smoothing calculation
this.smoothedBass += (rawBass - this.smoothedBass) * this.audioSmoothingFactor;
this.smoothedMid += (rawMid - this.smoothedMid) * this.audioSmoothingFactor;
this.smoothedTreble += (rawTreble - this.smoothedTreble) * this.audioSmoothingFactor;
```

Smoothing Guidelines: - **0-10%**: Very smooth, sluggish response (good for slow visualizations) - **10-20%**: Balanced, prevents jitter while staying responsive (recommended) - **20-40%**: More responsive, slight jitter on sudden changes - **40-100%**: Very responsive, may appear jumpy

---

Per-Feature Intensity Controls

Allow independent control of how strongly each feature reacts to audio:

```
// In constructor
this.colorIntensity = 1.0;   // 100%
this.speedIntensity = 1.0;   // 100%
this.segmentIntensity = 1.0; // 100%

// Add controls
this.addControl('colorIntensity', {
    type: 'dial',
    label: 'Color Intensity',
    min: 0,
    max: 200,
    step: 5,
    value: 100,
    onChange: (value) => {
        this.colorIntensity = value / 100;
    }
});

this.addControl('speedIntensity', {
    type: 'dial',
    label: 'Speed Intensity',
    min: 0,
    max: 200,
    step: 5,
    value: 100,
    onChange: (value) => {
        this.speedIntensity = value / 100;
    }
});

this.addControl('segmentIntensity', {
    type: 'dial',
    label: 'Segment Intensity',
    min: 0,
    max: 200,
```

```
    step: 5,
    value: 100,
    onChange: (value) => {
        this.segmentIntensity = value / 100;
    }
});
```

Applying Per-Feature Intensity

```
// Color reactivity
if (this.beatReact && this.audioColor) {
    const hueShift = audioData.energy * 120 * this.colorIntensity;
    primaryColor = this.shiftHue(colors.primary, hueShift);
}

// Speed reactivity
if (this.beatReact && this.audioSpeed) {
    const speedMultiplier = 1.0 + (audioData.energy * 0.5 * this.speedIntensity);
    finalRadius = baseRadius * speedMultiplier;
}

// Segment reactivity
if (this.beatReact && this.audioSegments) {
    const extraSegments = Math.floor(audioData.energy * 24 * this.segmentIntensity);
    sides = this.segmentCount + extraSegments;
}
```

Use Cases: - User wants color to react strongly but speed to react subtly - Fine-tuning the “feel” of audio reactivity per feature - Creating dramatic effects by cranking one intensity to 200%

Frequency-Specific Reactivity

Map specific frequency ranges (bass/mid/treble) to specific visual effects:

Bass → Rotation

```
// In constructor
this.bassRotation = false;

// Add toggle
this.addControl('bassRotation', {
    type: 'checkbox',
    label: 'Bass → Rotation',
    checked: false,
    className: 'btn-primary-mixer',
    onChange: (value) => {
        this.bassRotation = value;
    }
});

// Apply in onUpdate
onUpdate(deltaTime) {
    const audioData = this.getAudioData();

    if (this.beatReact && this.bassRotation) {
        this.rotation += audioData.bass * deltaTime * 5;
    }
}
```

Bass → Ring Spawn Rate

```
// In constructor
this.bassSpawn = false;
this.baseSpawnInterval = 0.3; // Base interval

// Add toggle
this.addControl('bassSpawn', {
    type: 'checkbox',
    label: 'Bass → Spawn',
    checked: false,
    className: 'btn-primary-mixer',
    onChange: (value) => {
        this.bassSpawn = value;
    }
});

// Apply in onUpdate
onUpdate(deltaTime) {
    const audioData = this.getAudioData();

    // Calculate spawn interval
    let spawnInterval = this.baseSpawnInterval;
    if (this.beatReact && this.bassSpawn) {
        // Higher bass = faster spawning (shorter interval)
        spawnInterval = this.baseSpawnInterval / (1 + audioData.bass * 2);
    }

    this.timeSinceLastRing += deltaTime;
    if (this.timeSinceLastRing >= spawnInterval) {
        this.spawnRing();
        this.timeSinceLastRing = 0;
    }
}
```

Mid → Glow Intensity

```
// In constructor
this.midGlow = false;

// Add toggle
this.addControl('midGlow', {
    type: 'checkbox',
    label: 'Mid → Glow',
    checked: false,
    className: 'btn-primary-mixer',
    onChange: (value) => {
        this.midGlow = value;
    }
});

// Apply in drawSegment
drawSegment() {
    const audioData = this.getAudioData();

    let glowAmount = this.glowIntensity * 20; // Base glow

    if (this.beatReact && this.midGlow) {
        glowAmount *= (1 + audioData.mid * 2); // Up to 3x glow on mids
    }

    this.ctx.shadowBlur = glowAmount;
    this.ctx.shadowColor = colors.primary;
}
```

Treble → Line Thickness

```
// In constructor
this.trebleThickness = false;

// Add toggle
this.addControl('trebleThickness', {
    type: 'checkbox',
    label: 'Treble → Thickness',
    checked: false,
    className: 'btn-primary-mixer',
    onChange: (value) => {
        this.trebleThickness = value;
    }
});

// Apply in drawSegment
drawSegment() {
    const audioData = this.getAudioData();

    let thickness = this.ringThickness; // Base thickness

    if (this.beatReact && this.trebleThickness) {
        thickness *= (1 + audioData.treble * 0.5); // Up to 1.5x on treble
    }

    this.ctx.lineWidth = thickness;
}
```

Frequency Mapping Best Practices: - **Bass** → Physical movement (rotation, spawning, pulsing) - **Mids** → Visual intensity (glow, brightness, saturation) - **Treble** → Fine details (thickness, particle count, shimmer)

Ring Spawning System

For continuous tunnel effects where rings spawn at the center, grow, and disappear at edges.

Why Use Ring Spawning?

Advantages: - Smooth, organic tunnel feel - Efficient (only active rings are rendered) - Easy to control speed and density - Natural lifespan management

Use Cases: - Tunnel visualizations (Tempest-style) - Ripple effects expanding from center - Particle systems with timed lifecycle - Wave/pulse effects

---

Basic Implementation

Constructor Setup

```
constructor(visualizer) {
    super('myplugin', visualizer, { /* ... */ });

    // Ring spawning system
    this.rings = []; // Array of { age: 0-1, rotation: angle }
    this.ringSpawnInterval = 0.3; // Spawn new ring every 0.3 seconds
    this.timeSinceLastRing = 0;
    this.ringLifetime = 3.0; // Each ring lives for 3 seconds

    this.setupControls();
    this.setupPresets();
}
```

Update Logic

```
onUpdate(deltaTime, timestamp, sharedAudioData) {
    this.time += deltaTime;
    const audioData = this.getAudioData();

    // Spawn new rings
    this.timeSinceLastRing += deltaTime;
    if (this.timeSinceLastRing >= this.ringSpawnInterval) {
        this.rings.push({
            age: 0,
            rotation: this.rotation // Capture current rotation
        });
        this.timeSinceLastRing = 0;
    }

    // Age existing rings
    for (let i = this.rings.length - 1; i >= 0; i--) {
        this.rings[i].age += deltaTime / this.ringLifetime;

        // Remove dead rings
        if (this.rings[i].age >= 1) {
            this.rings.splice(i, 1);
        }
    }
}
```

Render Logic

```
onRender(deltaTime, timestamp, sharedAudioData) {
    const audioData = this.getAudioData();
    const colors = this.getColors();

    // Clear background
    this.ctx.fillStyle = '#00003340';
    this.ctx.fillRect(0, 0, this.canvas.width, this.canvas.height);

    // Draw all active rings from oldest to newest (back to front)
    for (const ring of this.rings) {
        this.drawRing(ring.age, ring.rotation, colors, audioData);
    }
}
```

Drawing Individual Rings

```
drawRing(age, rotation, colors, audioData) {
    // age = 0 (just spawned) to 1 (about to die)

    // Scale from small to large based on age
    // At age=0: radius = 50 (center)
    // At age=1: radius = 800 (edge of screen)
    const minRadius = 50;
    const maxRadius = 800;
    const radius = minRadius + (age * (maxRadius - minRadius));

    // Fade out as ring ages
    const alpha = Math.max(0, 1 - age);

    // Audio-reactive size
    const audioScale = 1 + (audioData.bass * 0.3);
    const finalRadius = radius * audioScale * this.scale;

    const sides = this.segmentCount;

    this.ctx.save();
    this.ctx.translate(this.canvas.width / 2, this.canvas.height / 2);
    this.ctx.rotate(rotation); // Use ring's captured rotation

    // Glow
    if (this.glowIntensity > 0) {
        this.ctx.shadowBlur = 20 * this.glowIntensity;
        this.ctx.shadowColor = colors.primary;
    }

    // Draw polygon
    this.ctx.beginPath();
    this.ctx.lineWidth = this.ringThickness;

    for (let i = 0; i <= sides; i++) {
        const angle = (i / sides) * Math.PI * 2;
```

```
        const x = Math.cos(angle) * finalRadius;
        const y = Math.sin(angle) * finalRadius;

        if (i === 0) {
            this.ctx.moveTo(x, y);
        } else {
            this.ctx.lineTo(x, y);
        }
    }

    // Color gradient
    const gradient = this.ctx.createLinearGradient(
        -finalRadius, -finalRadius,
        finalRadius, finalRadius
    );
    gradient.addColorStop(0, colors.primary);
    gradient.addColorStop(0.5, colors.secondary);
    gradient.addColorStop(1, colors.tertiary);

    this.ctx.strokeStyle = gradient;
    this.ctx.globalAlpha = alpha;
    this.ctx.stroke();

    this.ctx.restore();
}
```

Audio-Reactive Ring Spawning

Bass-Controlled Spawn Rate

```
onUpdate(deltaTime) {
    const audioData = this.getAudioData();

    // Higher bass = faster spawning
    let spawnInterval = this.ringSpawnInterval;
    if (this.beatReact && this.bassSpawn) {
        spawnInterval = this.ringSpawnInterval / (1 + audioData.bass * 2);
    }

    this.timeSinceLastRing += deltaTime;
    if (this.timeSinceLastRing >= spawnInterval) {
        this.rings.push({ age: 0, rotation: this.rotation });
        this.timeSinceLastRing = 0;
    }

    // Age rings...
}
```

Beat-Triggered Bursts

```
onUpdate(deltaTime) {
    const audioData = this.getAudioData();

    // Detect beat (bass threshold)
    const beatThreshold = 0.7;
    const isBeat = audioData.bass > beatThreshold && !this.lastBeat;
    this.lastBeat = audioData.bass > beatThreshold;

    // Spawn burst of rings on beat
    if (isBeat) {
        for (let i = 0; i < 5; i++) {
            this.rings.push({
                age: i * 0.05, // Slight offset
                rotation: this.rotation + (i * Math.PI / 10)
            });
        }
    }

    // Normal spawning...
    // Age rings...
}
```

Performance Optimization

Ring Limit

Prevent too many rings from accumulating:

```
onUpdate(deltaTime) {
    const maxRings = 100;

    // Spawn new ring
    if (this.timeSinceLastRing >= this.ringSpawnInterval) {
        if (this.rings.length < maxRings) {
            this.rings.push({ age: 0, rotation: this.rotation });
        }
        this.timeSinceLastRing = 0;
    }

    // Age and remove...
}
```

Culling Off-Screen Rings

```
drawRing(age, rotation, colors, audioData) {
    const radius = 50 + (age * 750);

    // Skip if ring is off-screen
    const maxViewport = Math.sqrt(
        this.canvas.width * this.canvas.width +
        this.canvas.height * this.canvas.height
    );

    if (radius > maxViewport) {
        return; // Don't draw
    }

    // Draw ring...
}
```

Post-Processing Effects

Modern retro effects applied after rendering for authentic 80s/90s aesthetics.

Pixelation Effect

Create low-resolution retro look by downscaling and upscaling with nearest-neighbor filtering.

Setup

```
// In constructor
this.pixelCanvas = null;
this.pixelCtx = null;
this.pixelation = 0; // 0 = full res, 100 = maximum pixelation

// Add control
this.addControl('pixelation', {
    type: 'dial',
    label: 'Pixelation',
    min: 0,
    max: 100,
    step: 5,
    value: 0,
    onChange: (value) => {
        this.pixelation = value;

        // Create/destroy pixel buffer as needed
        if (value > 0 && !this.pixelCanvas) {
            this.pixelCanvas = document.createElement('canvas');
            this.pixelCtx = this.pixelCanvas.getContext('2d', {
                willReadFrequently: true
            });
        } else if (value === 0 && this.pixelCanvas) {
            this.pixelCanvas = null;
            this.pixelCtx = null;
        }
    }
});
```

Implementation

```
applyPixelation() {
    if (this.pixelation === 0) return;

    // Calculate target resolution
    // At 100%: 160x120 (classic retro)
    // At 50%: 320x240
    // At 25%: 640x480
    const pixelFactor = 1 - (this.pixelation / 100);
    const minResolution = 160;
    const targetWidth = Math.max(
        minResolution,
        Math.floor(this.canvas.width * pixelFactor)
    );
    const targetHeight = Math.max(
        Math.floor(minResolution * (this.canvas.height / this.canvas.width)),
        Math.floor(this.canvas.height * pixelFactor)
    );

    // Set pixel buffer size
    this.pixelCanvas.width = targetWidth;
    this.pixelCanvas.height = targetHeight;

    // Downscale to pixel buffer (smoothing OFF for sharp pixels)
    this.pixelCtx.imageSmoothingEnabled = false;
    this.pixelCtx.drawImage(this.canvas, 0, 0, targetWidth, targetHeight);

    // Clear main canvas
    this.ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);

    // Upscale back to main canvas (nearest-neighbor)
    this.ctx.imageSmoothingEnabled = false;
    this.ctx.drawImage(
        this.pixelCanvas,
        0, 0,
        this.canvas.width,
```

```
            this.canvas.height
        );

        // Re-enable smoothing for future operations
        this.ctx.imageSmoothingEnabled = true;
    }
```

Render Order

```
    onRender() {
        // 1. Draw all visual elements
        this.drawTunnel();

        // 2. Apply other effects (scanlines, static, etc.)
        this.drawScanlines();
        this.drawStatic();

        // 3. Apply pixelation LAST
        if (this.pixelation > 0) {
            this.applyPixelation();
        }
    }
```

Important: Pixelation must be applied LAST, after all other drawing is complete.

Color Banding (Posterization)

Reduce color depth for authentic 8-bit/16-bit retro look.

Setup

```
// In constructor
this.colorBanding = 0; // 0 = full color (24-bit), 100 = extreme banding (1-bit)

// Add control
this.addControl('colorBanding', {
    type: 'dial',
    label: 'Color Banding',
    min: 0,
    max: 100,
    step: 5,
    value: 0,
    onChange: (value) => {
        this.colorBanding = value;
    }
});
```

Implementation

```
applyColorBanding() {
    if (this.colorBanding === 0) return;

    // Get image data
    const imageData = this.ctx.getImageData(
        0, 0,
        this.canvas.width,
        this.canvas.height
    );
    const data = imageData.data;

    // Calculate color depth based on banding amount
    // 0%: 256 levels (8-bit per channel, full color)
    // 25%: 64 levels (6-bit per channel)
    // 50%: 16 levels (4-bit per channel)
    // 75%: 4 levels (2-bit per channel)
    // 100%: 2 levels (1-bit per channel, pure posterization)
    const bandingFactor = this.colorBanding / 100;
    const levels = Math.max(2, Math.floor(256 * (1 - bandingFactor * 0.992)));
    const step = 255 / (levels - 1);

    // Quantize each pixel
    for (let i = 0; i < data.length; i += 4) {
        data[i] = Math.round(data[i] / step) * step;         // Red
        data[i + 1] = Math.round(data[i + 1] / step) * step; // Green
        data[i + 2] = Math.round(data[i + 2] / step) * step; // Blue
        // Alpha (i + 3) unchanged
    }

    // Put modified data back
    this.ctx.putImageData(imageData, 0, 0);
}
```

Render Order

```
onRender() {
    // 1. Draw all visual elements
    this.drawTunnel();
    this.drawScanlines();
    this.drawStatic();
    this.drawRGBShift();

    // 2. Apply color banding BEFORE vignette/curvature
    //    This prevents posterizing smooth gradients
    if (this.colorBanding > 0) {
        this.applyColorBanding();
    }

    // 3. Apply vignette/curvature (smooth gradients preserved)
    this.drawVignette();
    this.drawScreenCurvature();

    // 4. Apply pixelation last
    if (this.pixelation > 0) {
        this.applyPixelation();
    }
}
```

Critical: Apply color banding BEFORE vignette/screen curvature to avoid posterizing those smooth gradients.

Enhanced RGB Shift

Improved chromatic aberration with vertical component and edge fringing.

Basic RGB Shift (Horizontal Only)

```
drawRGBShift() {
    if (this.rgbShift === 0) return;

    const audioData = this.getAudioData();
    const shiftAmount = this.rgbShift * 8 * (1 + audioData.treble);

    this.ctx.globalAlpha = 0.15 * this.rgbShift;
    this.ctx.globalCompositeOperation = 'screen';

    // Red shift left
    this.ctx.fillStyle = '#ff0000';
    this.ctx.fillRect(-shiftAmount, 0, this.canvas.width, this.canvas.height);

    // Blue shift right
    this.ctx.fillStyle = '#0000ff';
    this.ctx.fillRect(shiftAmount, 0, this.canvas.width, this.canvas.height);

    this.ctx.globalCompositeOperation = 'source-over';
    this.ctx.globalAlpha = 1;
}
```

Enhanced RGB Shift (Vertical + Edge Fringing)

```
drawRGBShift() {
    if (this.rgbShift === 0) return;

    const audioData = this.getAudioData();

    // Enhanced shift amounts
    const baseShift = this.rgbShift * 8;
    const audioBoost = 1 + (audioData.treble * 1.5);
    const shiftH = baseShift * audioBoost; // Horizontal
    const shiftV = (baseShift * 0.5) * audioBoost; // Vertical (half of horizontal)

    this.ctx.globalAlpha = 0.15 * this.rgbShift;
    this.ctx.globalCompositeOperation = 'screen';

    // Red channel (left and up)
    this.ctx.fillStyle = '#ff0000';
    this.ctx.fillRect(-shiftH, -shiftV, this.canvas.width, this.canvas.height);

    // Blue channel (right and down)
    this.ctx.fillStyle = '#0000ff';
    this.ctx.fillRect(shiftH, shiftV, this.canvas.width, this.canvas.height);

    // Green channel (vertical only, opposite)
    this.ctx.fillStyle = '#00ff00';
    this.ctx.fillRect(0, shiftV * 0.5, this.canvas.width, this.canvas.height);

    // Edge fringing (at high intensity + audio)
    if (this.rgbShift > 0.3 && audioData.treble > 0.5) {
        this.ctx.globalAlpha = 0.1 * this.rgbShift * audioData.treble;

        // Magenta fringe (Red + Blue)
        this.ctx.fillStyle = '#ff00ff';
        this.ctx.fillRect(shiftH * 0.5, 0, this.canvas.width, this.canvas.height);

        // Cyan fringe (Green + Blue)
```

```
        this.ctx.fillStyle = '#00ffff';
        this.ctx.fillRect(-shiftH * 0.5, 0, this.canvas.width, this.canvas.height);
    }

    this.ctx.globalCompositeOperation = 'source-over';
    this.ctx.globalAlpha = 1;
}
```

Film Grain Static

Authentic static with clustering and color noise.

Basic Static (White Noise)

```
drawStatic() {
    if (this.staticAmount === 0) return;

    const intensity = this.staticAmount;
    const pixelCount = Math.floor(intensity * 500);

    for (let i = 0; i < pixelCount; i++) {
        const x = Math.random() * this.canvas.width;
        const y = Math.random() * this.canvas.height;
        const brightness = Math.random();

        this.ctx.fillStyle = `rgba(${brightness * 255}, ${brightness * 255}, ${brightness
        this.ctx.fillRect(x, y, 1, 1);
    }
}
```

Enhanced Static (Clustered + Color Noise)

```
drawStatic() {
    if (this.staticAmount === 0) return;

    const audioData = this.getAudioData();
    const intensity = this.staticAmount * (0.5 + audioData.treble * 0.5);
    const pixelCount = Math.floor(intensity * 500);

    // Cluster seed for film grain effect
    const clusterSeed = Math.random() * 1000;

    for (let i = 0; i < pixelCount; i++) {
        const x = Math.random() * this.canvas.width;
        const y = Math.random() * this.canvas.height;

        // Perlin-like clustering
        const clusterFactor = Math.sin(x * 0.1 + clusterSeed) *
                              Math.cos(y * 0.1 + clusterSeed);
        const shouldDraw = Math.random() < 0.5 + (clusterFactor * 0.3);

        if (shouldDraw) {
            const brightness = Math.random();

            // 70% grayscale, 30% colored (VHS artifact)
            if (Math.random() < 0.7) {
                // Grayscale static
                this.ctx.fillStyle = `rgba(${brightness * 255}, ${brightness * 255}, ${br
            } else {
                // Colored noise
                const r = brightness * 255 * (0.5 + Math.random() * 0.5);
                const g = brightness * 255 * (0.5 + Math.random() * 0.5);
                const b = brightness * 255 * (0.5 + Math.random() * 0.5);
                this.ctx.fillStyle = `rgba(${r}, ${g}, ${b}, ${intensity * 0.7})`;
            }

            // Vary pixel size (1-2px)
```

```
            const size = Math.random() < 0.8 ? 1 : 2;
            this.ctx.fillRect(x, y, size, size);
        }
    }
}
```

Best Practice: Use immediate updates for properties where users expect instant feedback (geometry, materials). Use deferred updates for properties affecting motion or animation flow.

Performance Optimization

Cube Camera Performance

Resolution Trade-offs

```
// Ultra quality (expensive)
this.cubeRenderTarget = new THREE.WebGLCubeRenderTarget(4096, {...});

// High quality (balanced)
this.cubeRenderTarget = new THREE.WebGLCubeRenderTarget(2048, {...});

// Good quality (performance)
this.cubeRenderTarget = new THREE.WebGLCubeRenderTarget(1024, {...});
```

GPU Memory Usage: - 4096: ~402 MB per cube camera - 2048: ~100 MB per cube camera
- 1024: ~25 MB per cube camera

Update Frequency

```
// Update every frame (best quality, expensive)
this.cubeCamera.update(this.renderer, this.scene);

// Update every N frames (good balance)
if (this.frameCount % 2 === 0) {  // Every other frame
    this.cubeCamera.update(this.renderer, this.scene);
}
this.frameCount++;
```

Shared vs Per-Object Cube Cameras

```
// ✅ RECOMMENDED: Shared cube camera with dynamic positioning
// Cost: 1 cube camera update per frame × 6 render passes = 6 renders/frame
this.cubeCamera.position.set(avgX, avgY, avgZ);
this.cubeCamera.update(this.renderer, this.scene);

// ❌ EXPENSIVE: Per-object cube cameras
// Cost: N cube cameras × 6 render passes = 6N renders/frame
this.spheres.forEach(sphere => {
    sphere.cubeCamera.position.copy(sphere.mesh.position);
    sphere.cubeCamera.update(this.renderer, this.scene);
});
```

Geometry Optimization

Sphere Detail Levels

```
// Ultra quality (heavy)
const geometry = new THREE.SphereGeometry(size, 64, 64);  // 4096 faces

// High quality (balanced)
const geometry = new THREE.SphereGeometry(size, 32, 32);  // 1024 faces

// Good quality (performance)
const geometry = new THREE.SphereGeometry(size, 16, 16);  // 256 faces
```

Geometry Instancing

For many similar objects:

```
const geometry = new THREE.SphereGeometry(1, 32, 32);
const material = new THREE.MeshStandardMaterial({...});

const instancedMesh = new THREE.InstancedMesh(geometry, material, count);

// Set positions
for (let i = 0; i < count; i++) {
    const matrix = new THREE.Matrix4();
    matrix.setPosition(x, y, z);
    instancedMesh.setMatrixAt(i, matrix);
}

instancedMesh.instanceMatrix.needsUpdate = true;
this.scene.add(instancedMesh);
```

General Performance Tips

Minimize State Changes

```
// ❌ BAD: Change material every frame
this.spheres.forEach(sphere => {
    sphere.mesh.material.roughness = newValue;
    sphere.mesh.material.needsUpdate = true;
});

// ✅ GOOD: Only update when value actually changes
if (this.roughness !== newRoughness) {
    this.roughness = newRoughness;
    this.updateSphereMaterials();
}
```

Conditional Updates

```
// Only update when necessary
if (this.mappingMode === 'reflection' &&
    this.envMapSource === 'video' &&
    this.videoElement &&
    !this.videoElement.paused) {
    this.cubeCamera.update(this.renderer, this.scene);
}
```

Disposal

```
onCleanup() {
    // Dispose geometries
    this.spheres.forEach(sphere => {
        sphere.mesh.geometry.dispose();
        sphere.mesh.material.dispose();
    });

    // Dispose textures
    if (this.videoTexture) this.videoTexture.dispose();
    if (this.cubeMapTexture) this.cubeMapTexture.dispose();
    if (this.cubeRenderTarget) this.cubeRenderTarget.dispose();

    // Dispose renderer
    if (this.renderer) {
        this.renderer.dispose();
    }
}
```

Making Controls Actually Work (v2.5)

A control updating a variable isn’t enough - the variable must be USED in your rendering or update logic.

Example: Laser Frequency Control

Variable is updated:

```
this.addControl('laserFrequency', {
    type: 'dial',
    min: 0,
    max: 100,
    value: 30,
    onChange: (value) => {
        this.laserFreq = value;  // ✅ Variable updates
    }
});
```

But it must be USED in game logic:

```
updateLasers(deltaTime) {
    // ✅ CORRECT - laserFreq controls fire rate
    const baseFireChance = this.laserFreq / 1000;
    const audioBoost = 1 + (this.midLevel * 2);

    if (Math.random() < baseFireChance * audioBoost) {
        this.fireLaser();
    }
}
```

Example: Color Reactivity Control

Variable is updated:

```
this.addControl('colorReact', {
    type: 'dial',
    min: 0,
    max: 50,
    value: 20,
    onChange: (value) => {
        this.colorReactivity = value;  // ✅ Variable updates
    }
});
```

Must be used when getting colors:

```
getColor(index) {
    const baseColor = this.colors[index];

    // ✅ CORRECT - colorReactivity affects brightness
    const reactAmount = this.colorReactivity / 100;

    if (reactAmount > 0 && this.trebleLevel > 0) {
        return this.brightenColor(baseColor, this.trebleLevel * reactAmount);
    }

    return baseColor;
}
```

Verification Checklist

For each control you add: 1. ✅ Variable is initialized in constructor 2. ✅ Control value matches initial variable value 3. ✅ onChange updates the variable 4. ✅ Variable is actually USED in onUpdate() or onRender() 5. ✅ Effect is visible when control is changed

---

Audio Reactivity Best Practices (v2.5)

Making Controls Work at All Audio Levels

Problem: Controls Only Work With Loud Music

```
// ❌ WRONG - Only works when treble > 0.3 (loud music required)
if (this.colorReactivity > 0 && this.trebleLevel > 0.3) {
    this.applyColorEffect();
}

// ❌ WRONG - Only fires when beat detected
if (this.beatDetected && Math.random() < this.laserFreq / 100) {
    this.fireLaser();
}
```

Solution: Base Value + Audio Multiplier

```
// ✅ CORRECT - Works at any treble level > 0
if (this.colorReactivity > 0 && this.trebleLevel > 0) {
    const amount = this.colorReactivity / 100;
    this.applyColorEffect(this.trebleLevel * amount);
}

// ✅ CORRECT - Fires based on control, boosted by audio
const baseFireChance = this.laserFreq / 1000;  // Control sets base rate
const audioBoost = 1 + (this.midLevel * 2);     // Audio increases it
const beatBoost = this.beatDetected ? 3 : 1;   // Beats triple it

if (Math.random() < baseFireChance * audioBoost * beatBoost) {
    this.fireLaser();
}
```

Pattern: Base Value + Audio Multiplier

```
// Base behavior controlled by dial
const baseSpeed = this.speed;

// Audio multiplies the base
const audioMultiplier = 1 + (this.bassLevel * 2);

// Final value
const finalSpeed = baseSpeed * audioMultiplier;
```

Benefits: - ✅ Control works even with no audio (uses base value) - ✅ Audio enhances the effect (multiplies base value) - ✅ User always sees immediate feedback from controls - ✅ Effect scales naturally with music intensity

Audio Reactivity Mapping

Bass → Size/Scale:

```
const scale = 1.0 + (this.bassLevel * this.bassScaling);
```

Mid → Movement/Speed:

```
const speed = this.baseSpeed * (1 + this.midLevel * 2);
```

Treble → Brightness/Detail:

```
const brightness = 0.5 + (this.trebleLevel * 0.5);
```

Beat → Trigger Events:

```
if (this.beatDetected) {
    this.createExplosion();
}
```

Best Practices

Do’s

✅ Get dimensions from `visualizationContainer` ✅ Use individual CSS property assignment (not cssText) ✅ Set `width: 100%` and `height: 100%` for responsive sizing ✅ Use `devicePixelRatio` for high-DPI displays ✅ Use sRGB color space throughout ✅ Dispose resources in `onCleanup()` ✅ Use direct DOM queries for control updates ✅ Add comprehensive comments ✅ Test on different devices and browsers

Don’ts

❌ Use canvas buffer dimensions for renderer size ❌ Copy cssText from base canvas ❌ Use fixed pixel dimensions in CSS ❌ Forget to update video textures every frame ❌ Create per-object cube cameras without considering performance ❌ Forget to hide objects during cube camera capture ❌ Mix color spaces (LinearEncoding vs sRGBEncoding) ❌ Update cube cameras when not in reflection mode

Common Pitfalls

1. Canvas Sizing

Problem: Three.js canvas appears at wrong size

Solution: Get size from container, use percentage CSS

2. Video Reflections Not Working

Problem: Video doesn’t appear in chrome reflections

Solution: Implement cube camera system (required for Three.js r120+)

3. Upside-Down Textures

Problem: Video or images appear inverted

Solution: Use `flipY = true` instead of rotation

4. Color Mismatch

Problem: Video reflections look different from image reflections

Solution: Use consistent sRGB encoding throughout

5. Performance Issues

Problem: Frame rate drops with video reflections

Solution: Reduce cube map resolution, update less frequently, or use shared cube camera

---

Complete Example: Chrome Spheres with Video Reflections

See the Chrome Spheres plugin for a complete working implementation of: - Cube camera system - Dynamic positioning - Layer management - Dual mapping modes - Audio reactivity - Button toggle updates - Performance optimization

Study that plugin as the reference implementation for advanced Three.js techniques in Freque.

Common Plugin Development Mistakes (v2.5)

1. Forgetting className on Dropdowns

```
// ❌ WRONG - Dropdown won't display items
this.addControl('scheme', {
    type: 'dropdown',
    options: [{ value: 'red', label: 'Red' }]
    // Missing className!
});

// ✅ CORRECT - Displays properly
this.addControl('scheme', {
    type: 'dropdown',
    className: 'dropdown-selector-mixer',
    options: [{ value: 'red', label: 'Red' }]
});
```

2. Wrong Dropdown Options Format

```
// ❌ WRONG - Simple array won't work
options: ['Red', 'Green', 'Blue']

// ❌ WRONG - Object won't work
options: { red: 'Red', green: 'Green' }

// ✅ CORRECT - Array of objects
options: [
    { value: 'red', label: 'Red' },
    { value: 'green', label: 'Green' },
    { value: 'blue', label: 'Blue' }
]
```

3. Not Initializing Variables

```
// ❌ WRONG - Variables undefined until user touches controls
constructor() {
    this.setupControls();
}

// ✅ CORRECT - All variables initialized
constructor() {
    this.size = 50;
    this.speed = 1.0;
    this.colorScheme = 'classic';
    this.setupControls();
}
```

4. Controls That Don’t Affect Anything

```
// ❌ WRONG - Variable updates but isn't used
onChange: (value) => {
    this.unused = value;  // Never referenced in onUpdate/onRender!
}

// ✅ CORRECT - Variable is actually used
onChange: (value) => {
    this.speed = value;
}
// ... then in onUpdate():
this.position += this.speed * deltaTime;
```

5. Audio Requirements Too Strict

```
// ❌ WRONG - Only works with loud music
if (this.trebleLevel > 0.5) {
    this.applyEffect();
}

// ✅ CORRECT - Works at any audio level
if (this.trebleLevel > 0) {
    const intensity = this.trebleLevel;
    this.applyEffect(intensity);
}
```

6. Not Using Base + Multiplier Pattern

```
// ❌ WRONG - Effect disappears with no audio
const size = this.bassLevel * 100;  // 0 with no audio!

// ✅ CORRECT - Base value + audio boost
const size = 50 + (this.bassLevel * 100);  // 50-150
```

Summary Checklist

When creating a plugin, verify:

Audio Integration: - [ ] Use `this.getAudioEnergy()` for overall energy - [ ] Use `sharedAudioData.bass/mid/treble/beat` for frequency data - [ ] Initialize audio variables in constructor

Dropdown Controls: - [ ] Use `className: 'dropdown-selector-mixer'` - [ ] Format options as `[{ value: 'x', label: 'X' }, ...]` - [ ] Initialize dropdown variable in constructor - [ ] Use the variable in rendering logic

All Controls: - [ ] Initialize ALL control variables in constructor - [ ] Match control `value` with initialized variable - [ ] Verify onChange callback updates the variable - [ ] Confirm variable is USED in onUpdate() or onRender() - [ ] Test that changing control actually affects visualization

Audio Reactivity: - [ ] Base behavior works without audio - [ ] Audio enhances/multiplies base behavior - [ ] Don’t require specific audio thresholds to function - [ ] Provide immediate feedback when controls change

---

Additional Resources

Three.js Documentation

- Three.js Docs
- Three.js Examples
- CubeCamera
- WebGLCubeRenderTarget

Freque Resources

- Creating Plugins with Claude Guide
- Example Plugins in `js/plugins/`
- Freque Discord Community

Happy plugin development! 🚀🎨🎵

Guide Version 2.0 - Updated with cube camera system documentation

---